AI 原生软件研发体系
建设完全指南

企业实践版

从项目初始化到可验证交付

以 SQL 审核平台接入 GaussDB 为贯穿案例

适用读者　企业研发负责人　开发工程师　测试工程师　初学者

版本 1.0　　2026 年 10 月 11 日

阅读与操作说明

本手册帮助团队把 AI 辅助研发组织成一套可以复现、审查和交接的工作流程。完成学习后，你应能为项目建立清晰的入口规则和上下文文件，给每项需求准备完整任务包，按阶段调用产品、架构、开发、测试、审查和文档角色，并用实际代码、运行日志和负责人验收记录判断任务是否完成。落地时先选择一个范围清楚的小需求完成闭环，再根据试点数据扩大使用。

贯穿案例采用 Java、Spring Boot 和 Maven 的 SQL 审核平台，展示接入 GaussDB 的需求澄清、能力验证、适配器设计、测试和发布交接。目录、类名、任务编号及验收数据是教学设计；接入你的已有项目时，应先核对真实仓库结构、数据库产品、部署方式、版本与兼容模式，再调整模板。文中代码用于说明接口与控制路径，未接入经验证的解析器时必须返回不支持，不能据此宣称完整数据库支持。

先阅读第一至四章建立项目基础，再顺序完成第五至八章的案例。第九至十二章用于日常协作、知识维护、自动化和企业推广。每个步骤都配有操作说明、原因分析和产物要求；复制模板后要填入真实事实，保留未核实项。图示用于解释目录、界面和流程，其中界面图明确标为操作示意，具体按钮位置以安装版本为准。

自动目录的使用

本文件使用 Word 的标题样式和真正的目录域。点击目录中的条目可以跳转到相应章节；编辑章节、增删页面后，在目录上点击右键并选择“更新域”或“更新目录”，再选择“更新整个目录”。也可使用 Ctrl+A 后按 F9 更新域；部分笔记本需要同时按 Fn。导航窗格可按标题快速定位内容。目录中的页码以更新后的 Word 排版为准。

代码和文件模板以可选取的正文保存，便于复制到项目中。复制 PowerShell、YAML 或 Java 片段后，检查缩进、引号、路径与版本；不要用实际密码替换示例并提交到仓库。学习时先在练习分支或隔离的工作目录操作，用少量 SQL 语料验证一个规则，再扩展到复杂语法与真实数据库环境。

目录

阅读与操作说明 2

自动目录的使用 2

第一章 AI 原生研发基础 6

1.1 先定义交付对象，再选择 AI 参与方式 6

1.2 让上下文成为文件，而不是依赖对话记忆 6

1.3 建立需求到知识更新的交付闭环 6

1.4 把业务判断、执行权限和质量责任分开 7

1.5 用适用边界和小样本验证控制预期 8

第二章 项目初始化与目录构建 9

2.1 勘察已有仓库，建立可核实的基线 9

2.2 为新项目准备编辑与构建环境 9

2.3 按职责创建目录，保留现有源码布局 10

2.4 编写忽略规则，保护可复现资料与敏感配置 11

2.5 写清 README 与目录说明，形成第一次可用提交 12

2.6 在 VS Code 中完成一次读写与定位练习 13

第三章 项目大脑文件的编写与维护 14

3.1 用 AGENTS.md 建立可发现的规则入口 14

3.2 用 AI_CONTEXT.md 保存事实、入口与未知项 14

3.3 用 AI_RULES.md 把边界写成可执行条件 16

3.4 用 AI_WORKFLOW.md 定义阶段、门槛和失败返回 17

3.5 建立任务文件，避免聊天交接丢失 18

3.6 建立更新触发条件与资料核对习惯 19

第四章 六种 AI 角色与交接规范 21

4.1 产品 AI：把业务目标变成可测试需求 21

4.2 架构 AI：在既有扩展点中解释方案取舍 21

4.3 开发 AI：把方案拆成可审查的小批次修改 22

4.4 测试 AI：从验收条件推导验证，而不是复制实现 22

4.5 Review AI：围绕缺陷与证据进行独立审查 23

4.6 文档 AI：把本次发现提炼为下次可用知识 24

4.7 用 RACI 与交接包避免角色空转 24
第五章 需求澄清与架构设计 27
5.1 步骤一：确认目标产品、部署、版本与兼容模式 27
5.2 步骤二：把业务意图写成完整 PRD 28
5.3 步骤三：构建能力矩阵，限制支持声明的范围 28
5.4 步骤四：写架构方案和 ADR，记录取舍 29
5.5 步骤五：评审插件边界与修改清单 30
第六章 任务拆解与适配器实现 31
6.1 步骤一：按证据拆分任务，而不是按文件平均分工 31
6.2 步骤二：先定义审核状态和核心接口 32
6.3 步骤三：实现诚实返回 UNSUPPORTED 的适配器骨架 33
6.4 步骤四：逐项接入解析器与规则，控制修改范围 34
6.5 步骤五：维护 status.md，让任务可恢复 34
第七章 测试验证与代码审查 36
7.1 步骤一：建立 SQL 语料和人工确认的预期结果 36
7.2 步骤二：选择最小但有效的单元和契约测试 37
7.3 步骤三：分别执行旧目标回归与真实数据库集成 38
7.4 步骤四：分开评估误报、漏报和未覆盖 39
7.5 步骤五：让 Review AI 找反例，再由人负责结论 39
第八章 发布验收与完整交接 41
8.1 阶段一：需求确认与责任分配 41
8.2 阶段二：方案评审与风险边界冻结 41
8.3 阶段三：实现与持续记录 42
8.4 阶段四：测试与支持范围核验 42
8.5 阶段五：独立审查与发布决策 42
8.6 阶段六：灰度、观测与回滚演练 43
8.7 阶段七：业务验收与知识沉淀 43
8.8 阶段八：完整交接与下一轮任务启动 43
第九章 Codex 与 VS Code 日常操作 45
9.1 打开正确的项目根目录，并建立第一次检查点 45

9.2 核验 AGENTS.md 的作用范围，而不是凭感觉判断生效 46
9.3 用首轮勘察 Prompt 建立代码地图与任务范围 46
9.4 把会话进展写成可恢复的任务状态 47
9.5 分支、worktree 与多设备同步的日常纪律 47
9.6 工具中立迁移：把协议带到 OpenCode 或其他助手 48
第十章 项目知识的长期维护 49
10.1 建立知识索引与统一记录头 49
10.2 技术债不是愿望清单：记录影响、触发条件与偿还证据 50
10.3 事故库：先记录时间线，再追问为何防线失效 50
10.4 SQL 语料与数据库专家库：每个判断都带适用边界 51
10.5 ADR 模板与生命周期：保留做决定时的条件 51
10.6 防过期、去重复与隐私维护的固定动作 52
第十一章 CI 自动化与团队控制 53
11.1 先自动化可重复验证，再引入 AI 判断 53
11.2 GitHub Actions 最小可复制工作流与权限 54
11.3 Issue 到发布：每一次交接都带可核验材料 54
11.4 自动日报：从事实生成摘要，保持提交可追溯 55
11.5 评估质量、成本与安全，避免只计算生成速度 55
11.6 逐步升级自动化，并保留失败时的人工路径 56
第十二章 企业落地路线图与演练 57
12.1 试点范围、职责与量化验收标准 57
12.2 第一至三十天：把基础资料与验证跑通 58
12.3 第三十一至六十天：形成协作闭环与知识复用 58
12.4 第六十一至九十天：验证可复制，再决定推广 58
12.5 第一天完整演练：六个步骤逐一走完 59
12.6 排错、培训练习、FAQ 与总启动 Prompt 60
附录 官方资料与模板索引 62
A 官方资料与核对范围 62
B 核心模板与保存位置 62

第一章 AI 原生研发基础

1.1 先定义交付对象，再选择 AI 参与方式

AI 原生研发并不意味着把传统岗位名称全部改成 Agent，也不意味着把整个仓库一次性交给模型后等待成品。它的核心是把需求、设计、实现、验证和知识更新组织为可检查的协作过程，让 AI 在明确边界内承担适合的工作。对于 SQL 审核平台，业务价值不是“生成了多少代码”，而是用户提交一条 SQL 后，系统能够判断数据库类型、解释风险、给出可理解的建议，并且保留规则命中依据。企业应当先定义这样的交付对象，再讨论使用哪一种模型或工具。

本书以一个教学假定的 Java、Spring Boot 和 Maven 项目贯穿各章；该项目名称为 sql-audit-platform，Java 包名为 com.example.audit。这些名称只是为了让目录、提示词和测试案例保持一致，并不代表已经读取或了解读者现有项目。实际落地时，必须先检查真实代码、版本、部署环境和团队规范，再调整模板。第一项练习是把“让 AI 帮我开发”改写成可验收目标，例如“增加一种数据库的静态 SQL 审核能力，并保持已有 MySQL 与 TiDB 规则行为不变”。

表 1-1 从模糊指令到可验收任务

模糊表达	可执行表达	验收证据
帮我支持 GaussDB	确认具体产品与版本；在静态审核入口识别目标方言；输出可解释的规则结果	需求边界、SQL 样例、测试报告、代码审查记录
把代码优化一下	降低指定接口在既定数据集下的耗时，同时保留现有响应契约	基线记录、性能对比、回归结果
建立 AI 团队	建立六种角色的输入输出与交接规则，完成一个任务闭环	角色提示词、任务目录、交接文档

1.2 让上下文成为文件，而不是依赖对话记忆

大模型能够利用当前输入和工具读取到的信息，但这不等于拥有可靠、永久、完整的项目记忆。长对话可能被压缩，新的会话未必继承旧会话，工具访问范围也可能不同。如果重要约束仅出现在一周前的聊天里，下一次开发就可能遗漏数据库兼容要求或误解模块职责。团队应当把稳定事实写入版本管理中的文件，把本次任务进度写入任务目录，再在每次开工时明确要求 AI 读取相关内容。这样，同事、另一台电脑和新的 AI 会话都能够从同一组资料恢复工作。

项目上下文也不是越多越好。把所有历史日志、生成文件和会议记录同时塞入提示词，会增加干扰，并掩盖真正影响实现的约束。建议采用分层读取：先读根目录 AGENTS.md 与项目总览，再根据任务读相关架构决策和模块代码，最后读数据库资料与 SQL 样例。对于无法确认的事实，要求 AI 在任务状态中标为“待核实”，附上需要检查的位置。资料之间出现冲突时，不让模型自行猜测，应依据当前代码、测试和已批准决策进行核对，必要时由对应负责人作出判断。

1.3 建立需求到知识更新的交付闭环

一项任务至少要回答三个问题：我们准备改变什么，改变是否满足要求，其他人怎样复现验证。产品阶段写清输入、输出和例外情况；架构阶段解释扩展点与兼容性；实现阶段保持修改范围可控；

验证阶段提供运行环境、执行方式与结果。最后把新发现的限制写入知识库。流程中的文档不必都很长，但必须能够解释关键选择，不能用一句“已经完成”代替可检查的证据。对高风险模块，交付门槛还应覆盖回退方法、访问权限和审计记录。

图 1-1 展示了本书采用的闭环。箭头代表成果交接，而不是要求每一项工作都由独立模型完成；同一个 AI 会话可以承担多个角色，但切换角色时仍要使用新的输入清单和检查目标。任何阶段发现需求不清楚或验证失败，都应返回对应阶段修正，并更新状态文件。团队可以从一个小任务开始实践，例如给已有审核结果增加“命中规则标识”，用一轮完整闭环检查模板是否有用，再逐步扩展到数据库适配和自动化任务。

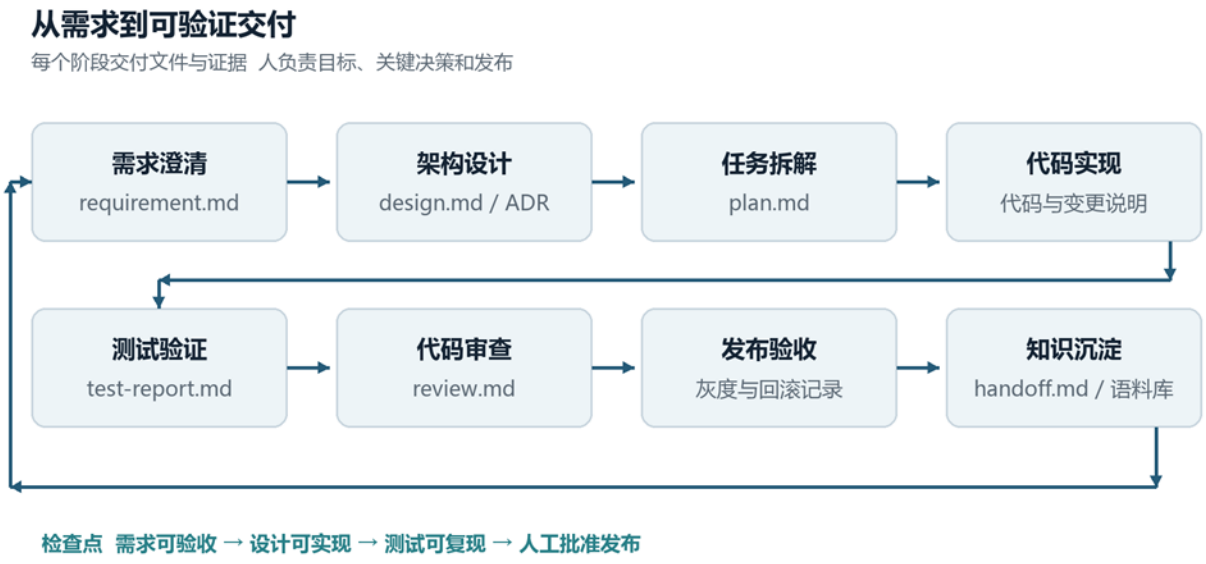


图 1-1 AI 原生研发闭环：需求、设计、实现、测试、审查与知识回写

表 1-2 每个阶段必须留下的证据

阶段	核心问题	最低交付物
需求	改变什么，哪些内容不在范围内	requirement.md 与验收条件
设计	沿用哪些扩展点，有哪些取舍	design.md 与必要的 ADR
实现	具体修改了哪些文件	代码差异与 plan.md 执行记录
测试	哪些行为被验证，哪些未验证	test-report.md 与可复现命令
审查	是否存在阻断问题	review.md 与问题处理记录
交接	下一个人如何继续	handoff.md 与 status.md

1.4 把业务判断、执行权限和质量责任分开

AI 可以提出方案、生成代码、设计测试和解释差异，但企业责任不能因为工具参与而消失。业务负责人应确定需求优先级、可接受风险和上线范围；技术负责人应确认架构方向与依赖策略；具体开发者应对交付内容和验证记录负责。模型自信的表述不是批准记录，工具返回成功也只说明某个操

作成功执行，不说明业务目标已经实现。团队尤其需要区分“允许起草”“允许修改代码”“允许部署”和“允许访问生产数据”四类权限，避免用一次笼统授权覆盖不同风险的动作。

执行边界应尽量写成可判断的规则。例如，允许 AI 在本地任务分支修改适配器与测试；新增依赖必须说明用途并由技术负责人批准；生产数据库连接、数据导出与正式发布遵循企业已有审批流程。对于可逆的低风险变更，负责人可以预先给出明确授权，减少无意义的反复确认。对于难以撤销的动作，则应先准备完整、可评审的方案与回退路径，再获得授权。这样的安排既让 AI 持续推进已授权工作，也使关键决策保留清晰的责任人。

1.5 用适用边界和小样本验证控制预期

AI 通常适合处理资料整理、局部实现、测试补充、代码解释和已有模式的扩展，但在需求矛盾、文档过期或数据库语义复杂时容易给出看似合理的错误结论。SQL 审核尤其包含方言差异、权限环境和执行计划等细节，静态分析不能证明真实执行成本，也不能保证业务语义正确。团队应把模型输出视为候选成果，用代码阅读、自动化测试和必要的环境验证确认。若任务依赖内部资料或特定商业产品版本，首先确保来源可用，而不是让 AI 补全缺失事实。

首次建设体系时，可以选择影响范围小、验收条件清晰的任务做试点，并记录从需求到交付的耗时、返工原因与漏检问题。比较时不要只统计模型生成速度，还要统计人类审查时间、测试可靠性、维护负担和故障恢复成本。至少完成两到三类不同任务后，再判断哪些角色值得拆分、哪些模板需要精简。采用这套方法的目的，是形成稳定的工程习惯；团队完全可以根据规模合并角色和文档，但不能取消需求边界、验证证据与交接责任。

第二章 项目初始化与目录构建

2.1 勘察已有仓库，建立可核实的基线

如果读者已经有 SQL 审核平台，第一步必须是勘察现有仓库，不能照着本书目录重新创建另一套工程。打开根目录后，先阅读 README、构建配置、现有 AGENTS.md、模块结构和测试入口，再确认当前工作区是否存在未提交修改。已有目录命名、Java 版本和构建工具优先于教学示例。让 AI 输出“已确认事实、待核实事项、建议读取文件”三部分报告，并引用文件路径；没有证据的技术栈、数据库版本和部署方式应保持未知。

下方 PowerShell 命令用于查看而非改写项目。执行前把路径替换为真实仓库位置，并确认当前目录是目标项目。对于单仓多模块工程，需要进一步检查各模块的 pom.xml 和嵌套规则；对于没有 Git 的目录，先确认这是否是从别处复制的代码以及团队希望如何保存。不要在勘察阶段自动升级依赖、移动源文件或覆盖配置。基线完成后，将真实结构与本书示例之间的差异记录到 AI_CONTEXT.md，后续所有提示词都以这个结果为准。

```
Set-Location -LiteralPath 'D:\Projects\sql-audit-platform'
Get-Location
Get-ChildItem -Force
git status --short
git branch --show-current
Get-Content -LiteralPath '.\README.md' -Encoding UTF8
Get-Content -LiteralPath '.\pom.xml' -Encoding UTF8
git ls-files
```

表 2-1 项目勘察记录

检查项	需要确认的内容	记录位置
业务入口	审核接口、输入 DTO、结果模型、规则执行入口	.ai/AI_CONTEXT.md
构建方式	JDK 版本、Maven Wrapper、测试命令、模块关系	.ai/AI_CONTEXT.md
现有变化	未提交文件、任务分支、不可覆盖的本地改动	.ai/tasks/GAUSS-001/status.md
规则与限制	已有 AGENTS.md、团队规范、依赖审批规则	AGENTS.md 及相关规则
数据库支持	适配器位置、真实产品版本、既有回归样例	.ai/database 及任务需求

2.2 为新项目准备编辑与构建环境

对于从零开始的练习，先准备可编辑文本的工具、Git 和与项目一致的 JDK 及 Maven 环境。VS Code 负责浏览目录、编辑文件和打开终端；Java 与 Maven 负责实际构建，它们的版本必须与 pom.xml 和团队环境一致。软件版本会变化，本书不指定必须使用某个最新版本，也不把安装成功等同于项目能够构建。若企业已经提供标准开发镜像、内网安装包或 Maven Wrapper，应优先采用这些材料，避免个人电脑自行组合出不受支持的环境。

在 VS Code 中使用“文件→打开文件夹”选择项目根目录，然后使用“终端→新建终端”打开 PowerShell。图 2-1 中的目录应出现在左侧资源管理器，根目录文件与 src 位于同一级。运行下方检查命令，确认工具能够被终端找到，并查看实际版本；如果项目有 mvnw.cmd，优先用它调用团队规定的 Maven。连接企业代码仓库或获取依赖失败时，先检查网络、认证和代理配置，不能把失败伪装成测试通过，也不要把账号口令写入项目文档。

```
git --version
java -version
mvn -version
# 若已有 Maven Wrapper，以项目实际文件为准:
.\mvnw.cmd -version
```

2.3 按职责创建目录，保留现有源码布局

下面的目录树是单模块教学工程的目标结构。src/main/java 存放生产代码，src/test/java 存放测试代码，src/test/resources/sql 存放被测试使用的 SQL 样例；.ai 用于保存项目协作材料。目录划分必须服务于检索和维护：任务材料按任务编号集中，架构决策按决策记录保存，数据库知识与实际案例分开，以便区分一般规则和某次故障的证据。已有项目如果采用不同包名或测试位置，应修改模板指向现有位置，不能为满足示例而进行大规模迁移。

PowerShell 命令使用显式路径列表创建目录，适合尚未建立结构的练习项目。创建之前先确认 Get-Location 显示目标根目录，随后执行并在 VS Code 资源管理器中刷新。Git 不会跟踪空目录；真正提交时，需要在目录中放入有意义的 README、模板或测试样例。仅为占位可以使用.gitkeep，但它只是约定名称，不具备 Git 特殊功能。本书建议知识目录先放一个说明文件，解释目录用途与收录标准，减少看见空目录却不知道如何使用的情况。



图 2-1 项目目录与文件定位操作示意

```
sql-audit-platform/
├─ AGENTS.md
├─ README.md
```

```
├ .gitignore
├ pom.xml
├ .ai/
│  ├ AI_CONTEXT.md
│  ├ AI_RULES.md
│  ├ AI_WORKFLOW.md
│  ├ architecture/adr/
│  ├ knowledge/
│  ├ database/
│  ├ sql-case/
│  ├ prompts/
│  └ tasks/GAUSS-001/
│     ├ requirement.md
│     ├ design.md
│     ├ plan.md
│     ├ status.md
│     ├ test-report.md
│     ├ review.md
│     └ handoff.md
├ src/main/java/com/example/audit/
├ src/test/java/com/example/audit/
└ src/test/resources/sql/
```

```
$taskDirectories = @(
  '.ai\architecture\adr', '.ai\knowledge', '.ai\database',
  '.ai\sql-case', '.ai\prompts', '.ai\tasks\GAUSS-001',
  'src\main\java\com\example\audit',
  'src\test\java\com\example\audit',
  'src\test\resources\sql'
)
foreach ($taskDirectory in $taskDirectories) {
  New-Item -ItemType Directory -Path $taskDirectory -Force | Out-Null
}
Get-ChildItem -LiteralPath '.ai' -Recurse -Force
```

2.4 编写忽略规则，保护可复现资料与敏感配置

版本控制要保留可复现的源码、规则与知识材料，同时排除构建产物、本地缓存和敏感配置。.gitignore 是给 Git 的忽略规则，不是访问控制，也不会自动删除已经提交的文件。编写时从已有规则出发，采用具体且容易理解的模式，避免直接忽略整个.ai 或全部 SQL 文件；这些恰好是协作体系需要保存的内容。对包含内部结构或客户数据的样例，应先脱敏再提交，并把真实凭据存放在企业认可的密钥管理方式中。

下方模板适用于本书练习工程，可以复制到项目根目录后按实际情况调整。application-local.yml 被视为本机配置，项目另行提供不含口令的示例文件，说明必需字段。创建完成后使用 git status 检查本该保存的材料是否出现，再用 git check-ignore 确认特定路径为什么被忽略。若发现真实密钥已经进入 Git 历史，只修改.gitignore 并不能修复泄露，需要按企业流程撤销密钥并处理历史记录。这里的学习重点是让共享材料完整，同时让本地配置与凭据有明确存放边界。

```
# Maven 构建产物
target/
# 编辑器与系统临时文件，按团队需要保留共享设置
*.swp
*.tmp
.DS_Store
Thumbs.db
# 本地环境与个人配置
.env
.env.*
!.env.example
src/main/resources/application-local.yml
# 本地日志与运行数据
*.log
logs/
# 本地 AI 工具缓存：仅在工具实际产生此目录时使用
.ai/local-cache/
# .ai 中的任务、规则、知识与 SQL 案例需要提交，不要忽略.ai/
```

```
git status --short
git check-ignore -v -- 'src/main/resources/application-local.yml'
git check-ignore -v -- '.ai/AI_CONTEXT.md'
# 最后一项无输出通常表示未被忽略；
# 仍需结合 git status 和现有 Git 状态判断。
```

2.5 写清 README 与目录说明，形成第一次可用提交

README 是人和 AI 都容易找到的项目入口，应回答项目做什么、如何构建、如何验证以及协作资料在哪里。它不应堆满规划性口号，也不应写尚未运行成功的启动命令。新建练习项目如果暂时只有目录和模板，就明确写“当前为初始化资料，应用尚未实现”；已有项目则保留真实运行方法。在每个知识目录放置简短说明，规定什么内容可以收录、如何引用来源、何时需要更新，使目录成为工作规则的一部分而不是装饰。

第一次提交前，检查根目录是否出现本章需要的文件，检查.gitignore 是否把本机配置排除，然后只暂存确认可以共享的文件。不要在未知仓库中直接使用笼统操作覆盖已有变更，也不要擅自创建与团队流程冲突的分支。下面示例假定这是读者新建的本地练习目录；执行 git init 后仍然需要查看差异再提交。提交消息应说明此次建立了协作骨架，而不是声称已实现审核功能。这样，未来回顾时可以准确区分项目准备、实际功能与质量验证三个层次。

```
# SQL 审核平台（教学练习）

## 项目用途
本项目用于演示 SQL 静态审核平台的 AI 协作流程。
当前状态：初始化资料已建立；应用实现与真实环境验证尚未完成。

## 技术基线
Java、Spring Boot、Maven。具体版本以 pom.xml 和实际运行环境为准。
教学包名：com.example.audit。

## 常用操作
已有 Maven Wrapper 时执行：.\mvnw.cmd test。
没有 Wrapper 时执行：mvn test。
命令是否成功必须以真实运行结果为准。

## AI 协作入口
先阅读根目录 AGENTS.md，再根据任务显式读取.ai 中的材料。
```

任务示例：.ai/tasks/GAUSS-001/。

配置与数据
不得提交数据库密码、访问令牌或客户原始数据。
环境字段应提供脱敏示例；SQL 样例需注明目的和预期结果。

贡献流程
确认需求与范围→设计→局部实现→验证→审查→更新任务交接。

```
# 仅用于尚未建立 Git 仓库的本地练习目录
git init
git status --short
git add -- AGENTS.md README.md .gitignore .ai
git diff --cached --stat
git diff --cached
git commit -m "docs: initialize AI collaboration structure"
```

2.6 在 VS Code 中完成一次读写与定位练习

目录建好后，应立即做一次完整的使用练习，确认工具能够找到材料。先在 VS Code 左侧单击 AGENTS.md，检查文件位于根目录且编码为 UTF-8；再展开.ai/tasks/GAUSS-001 并打开 status.md。通过编辑器的“转到文件”功能定位 AI_CONTEXT.md，填写已确认的业务与技术事实。把“还不知道具体 GaussDB 产品版本”保留为待核实事项，而不是复制未经确认的版本号。保存后查看源代码管理面板，确认修改涉及的文件与预期一致。

随后在所使用的 AI 工具里发送下方提示词，要求它按路径读取并复述边界。界面中的附加文件、引用文件或工具读取入口随产品版本变化，实际操作以当前工具文档为准；如果无法直接访问仓库，就复制相关内容并注明路径和时间。练习的验收标准是 AI 能够指出项目用途、禁止事项和当前任务状态，并承认尚未确认的信息。若它没有真正读到文件，继续增加对话篇幅也不能解决问题，应先修复文件访问或输入方式，然后再进入开发。

```
这是一个 SQL 审核平台教学项目。
请先读取根目录 AGENTS.md，再显式读取：
.ai/AI_CONTEXT.md
.ai/AI_RULES.md
.ai/AI_WORKFLOW.md
.ai/tasks/GAUSS-001/status.md

请只输出：
1. 你实际读到的文件清单；
2. 三项已确认的项目事实及其来源路径；
3. 当前任务范围、禁止事项与待核实事项；
4. 开始开发前仍然缺少的材料。
此轮不修改源码，不把未读到的内容当作已知事实。
```

第三章 项目大脑文件的编写与维护

3.1 用 AGENTS.md 建立可发现的规则入口

根目录 AGENTS.md 应当简短而明确，承担“工作入口与约束索引”的作用。Codex 具备发现 AGENTS.md 的机制，具体作用范围、嵌套目录的覆盖关系与发现限制应按官方文档理解[S1]。这种机制并不意味着任意.ai 文件都会自动读入，更不意味着把资料放进去就形成永久记忆。应在入口中指明需要读取的项目资料，并要求 AI 在任务开始时报告实际读取情况。对其他 AI 工具，也要检查它是否支持同样的约定，不能把一个产品的行为泛化到所有工具。

下方模板把常用构建命令、任务边界和资料路径写在入口中，便于新会话迅速开始。实际项目已有 AGENTS.md 时，需要合并和保留有效规则，不能直接覆盖。大型仓库可以按模块增加局部规则，但应避免在多处重复同一事实，因为重复资料容易出现矛盾。模板中的命令必须结合真实项目验证；如果构建工具不同，应替换对应命令。维护者可以指定为技术负责人，开发者遇到过期规则时提出修正，并用评审记录解释规则变化。

```
# 项目协作入口

## 适用范围
本文件位于仓库根目录。开展工作前检查相关子目录是否存在局部规则。
本项目示例包名为 com.example.audit；真实结构以当前代码为准。

## 开始任务
1. 读取 README.md 与.ai/AI_CONTEXT.md。
2. 显式读取.ai/AI_RULES.md、.ai/AI_WORKFLOW.md。
3. 若任务编号为 GAUSS-001，读取.ai/tasks/GAUSS-001/中的需求、设计、计划和状态。
4. 报告实际读取文件；将未确认事实标记为待核实。

## 修改范围
保留已存在的用户改动。先检查工作区与相关模块。
遵循任务计划；扩展范围前记录原因并按团队决策流程处理。
不得提交真实凭据、客户原始数据或未脱敏日志。

## 验证
已有 Maven Wrapper 时优先执行.\mvnw.cmd test；否则以 README 规定的命令为准。
记录真实执行命令、结果、未执行项与原因，不得虚构成功。

## 交付
更新任务 status.md、test-report.md、review.md 与 handoff.md。
引用修改文件与验证证据；需要长期保留的决策另写 ADR。
```

3.2 用 AI_CONTEXT.md 保存事实、入口与未知项

AI_CONTEXT.md 保存项目当前事实，不承担所有历史知识的集合功能。它应该让第一次进入项目的人迅速知道系统用途、主要模块、技术基线、外部系统和关键入口，同时看见有哪些信息尚未确定。推荐对重要事实注明依据，例如“审核入口以某个 Controller 和对应测试为准”，而不是只写抽象说明。教学模板没有真实类名时，明确保留待填写项，防止后续 AI 把模板里的占位内容当成已经存在的代码。事实文件越准确，任务分析就越少依赖猜测。

维护时遵循“影响项目理解的变化才更新总览”的原则。新增数据库适配、改变审核结果模型或升级构建基线，需要更新；修复一条普通测试则通常只写任务记录。产品负责人确认业务范围，技术

负责人确认结构与版本，任务执行者提交对应修改。每次更新写上日期和依据，避免总览与代码长期漂移。图 3-1 展示读取层次：入口指向事实与规则，事实指向模块，任务材料再指向本次需要的知识和证据。这样既方便恢复上下文，也能限制单次输入规模。



图 3-1 项目上下文分层读取

```
# AI_CONTEXT：项目事实总览

最后核对日期：待填写
维护责任人：技术负责人；业务范围由产品负责人确认
状态说明：以下为教学模板，未核实内容不得当作真实项目事实。

## 业务目标
提供 SQL 静态审核，输出风险级别、规则标识、解释与建议。
静态审核不连接生产数据库执行用户提交的 SQL。

## 技术基线
Java / Spring Boot / Maven。
实际版本、依赖来源与构建方式：以 pom.xml 和 README 核对后填写。
教学包名：com.example.audit。

## 主要目录
生产代码：src/main/java/com/example/audit/
测试代码：src/test/java/com/example/audit/
SQL 样例：src/test/resources/sql/
任务材料：.ai/tasks/<任务编号>/
架构决策：.ai/architecture/adr/

## 关键代码入口
HTTP 入口：待核对真实路径
审核服务：待核对真实路径
数据库适配器与注册机制：待核对真实路径
规则与结果模型：待核对真实路径

## 已支持数据库
待核对真实代码与测试；不得仅凭文档推断支持范围。

## 当前任务
GAUSS-001：评估并增加目标 GaussDB 产品的静态审核支持。
```

产品名称、部署形态、版本与方言差异：待确认。

重要限制
保留既有结果契约；不默认执行 SQL；不提交凭据与原始业务数据。

待核实清单
- 项目实际 JDK 与 Maven 版本；
- 适配器扩展机制；
- 目标数据库具体产品和版本；
- 可用测试环境与验收负责人。

更新记录
日期 | 事实变化 | 依据文件或已批准决策 | 修改人

3.3 用 AI_RULES.md 把边界写成可执行条件

AI_RULES.md 应当描述可判断的工程规则，包括修改范围、依赖引入、错误处理、测试真实性和敏感数据约束。诸如“代码要优雅”“不要出错”无法作为交付门槛，因为不同人对这些词有不同理解。更有效的写法是“未知数据库类型必须返回清晰错误，不能静默回退到 MySQL”，或者“新增规则必须提供命中与不命中的代表样例”。规则应解释触发条件和预期结果，让开发者、测试者和审查者能够使用同一个标准。

规则不宜将所有工作都变成审批流程。团队可以事先授权小范围、可逆的代码修改，同时对生产访问、契约改变和依赖变更保留明确的决策人。规则与真实需求冲突时，执行者要指出冲突及其影响，通过已有团队流程更新规则或任务边界，不能把自己的解释隐藏在代码里。推荐将稳定规则保存在总文件，将特定模块细则放在对应目录或任务设计中，并定期删除已经失效的限制，避免不断累积让正常工作无法推进。

AI_RULES：工程边界与质量规则

事实与证据
- 不将教学模板、模型推测或旧对话当作已确认事实。
- 引用当前文件路径；无法验证的结论列入待核实。
- 测试报告区分通过、失败、未运行与环境不可用。

范围控制
- 保留当前工作区已有修改。
- 按任务计划修改相关模块，避免无关格式化和重构。
- 结果契约、架构方向与新增依赖按团队既有流程确认。
- 本地授权不代表生产访问或正式发布授权。

SQL 审核行为
- 不执行用户提交的 SQL 来“验证安全性”。
- 未知数据库类型返回明确错误，不静默回退。
- 不支持或无法解析的语法必须明确标记，不能冒充审核通过。
- 规则结果至少说明规则标识、级别与原因；字段以现有契约为准。

验证与兼容
- 新增规则覆盖代表性命中与不命中样例。
- 数据库扩展需要检查既有 MySQL 与 TiDB 行为。
- 只在真实运行后填写成功；保留失败原因与剩余限制。

数据与凭据
- 不提交数据库口令、访问令牌和客户原始数据。
- SQL、日志与截图先脱敏；来源和授权范围可追溯。

```
## 维护
负责人：技术负责人。
更新触发：规则漏洞、契约变化、事故复盘或已批准流程变化。
更新时说明理由与相关任务编号。
```

3.4 用 AI_WORKFLOW.md 定义阶段、门槛和失败返回

工作流文件的价值不是画一条漂亮箭头，而是让执行者知道此时能够做什么、还缺什么，以及失败之后回到哪里。每个阶段需要明确输入、输出和交接门槛。需求未确认时可以继续勘察代码和收集样例，但不能把未知数据库版本写成既定实现目标；测试失败时可以定位修复，但不能把失败任务标成完成。通过这些条件，团队能够保持自主推进，同时避免在关键事实缺失时悄悄作出业务承诺。

下方模板适合日常功能开发。紧急修复可以压缩文档形式，但需要保留问题描述、范围、验证证据与回退说明。小任务可以在同一会话中依次切换角色，大任务则用明确的任务文件进行交接。状态记录不应只写“进行中”，还要写最近完成的事项、当前阻塞和下一步。阶段结束时检查是否真的满足门槛；如果未满足，标记剩余工作并继续处理，而不是因为代码已经生成就推动合并或发布。

```
# AI_WORKFLOW：任务闭环

## 0. 勘察
输入：任务描述、当前仓库、已有规则。
输出：已确认事实、相关模块、未知项与工作区状态。
门槛：知道本次范围和不可覆盖内容。

## 1. 需求
输入：业务目标、目标用户、数据库产品信息。
输出：requirement.md，包含范围、例外、验收条件与未决项。
门槛：验收条件可验证；未决项不阻碍当前授权工作。

## 2. 设计
输入：需求、相关代码、架构记录、数据库资料。
输出：design.md；有长期取舍时增加 ADR。
门槛：扩展点、兼容性、错误处理与回退方法明确。

## 3. 计划与实现
输入：已确认设计。
输出：plan.md、局部代码与 status.md。
门槛：修改范围符合计划；自检完成；剩余项记录完整。

## 4. 测试
输入：需求、代码差异、样例与执行环境。
输出：test-report.md，记录命令、环境、结果、未验证项。
门槛：验收条件有证据；失败已修复或按团队流程处理。

## 5. 审查
输入：需求、设计、差异与测试报告。
输出：review.md，按严重性列出可复现问题。
门槛：阻断问题关闭；剩余风险有明确决定与责任人。

## 6. 交接与沉淀
输入：最终差异、验证与审查结果。
输出：handoff.md；按需更新上下文、ADR 与知识库。
门槛：下一位执行者能够找到证据并继续工作。
```

```
## 返回规则
需求变化回到需求；设计缺陷回到设计；
实现或测试失败回到实现；来源冲突先核实事实。
状态更新必须反映真实进度，不能虚构完成。
```

3.5 建立任务文件，避免聊天交接丢失

任务目录是一次交付的工作账本，与长期知识库承担不同职责。GAUSS-001 目录中的 requirement.md 写目标与验收，design.md 写实现方案，plan.md 写拆解步骤，status.md 写实时进度，test-report.md 写验证，review.md 写审查，handoff.md 写交接。这些文件可以从简，但不应全部复制相同内容。需求变化时先更新需求与设计，再调整计划；测试报告应链接具体样例和代码，而不是只写数量。这样，新的会话能够恢复任务状态，不必追溯完整聊天历史。

下方状态与交接模板可以立即使用。每次结束工作前，执行者填写最后核对时间、已完成项、剩余问题和下一步，使状态足够新鲜。未运行的测试保留为未运行，未确认的产品版本保留为待核实，不通过措辞包装消除事实上的空缺。文件更新应与相关代码一起检查并提交，避免交接描述指向旧版本。任务完成后保留目录作为追溯证据；真正具有复用价值的结论再提炼到知识库，防止把全部过程记录堆进长期上下文。

表 3-1 任务文件的内容与维护责任

文件	必须包含	主要维护者
requirement.md	目标、范围、数据库标识、验收与未决项	产品角色及业务负责人
design.md	现状、扩展点、错误处理、兼容、回退	架构角色及技术负责人
plan.md	修改文件、步骤、依赖、完成条件	开发角色
status.md	当前阶段、完成项、未知项、下一步	当前执行者
test-report.md	环境、命令、结果、证据、未验证项	测试角色
review.md	审查范围、发现、严重性、处理结果	Review 角色
handoff.md	最终状态、关键路径、剩余风险、接手指引	开发与文档角色

```
# GAUSS-001 状态记录

最后核对时间：待填写
当前阶段：需求澄清
当前执行者：待填写
对应代码版本：待填写分支与提交；有未提交变化时明确标注

## 已完成
- 建立任务目录与协作入口。

## 待核实
- 目标 GaussDB 具体产品、部署形态与版本。
- 当前适配器注册机制。
- 可用验证环境。

## 当前阻塞
填写实际阻塞；没有则写“无”，不得把未知项自动视为已解决。
```

```
## 下一步
读取相关代码并补全需求的兼容范围与验收条件。

## 最近验证
记录真实命令、日期与结果；尚未执行时写"未执行"。
```

```
# GAUSS-001 交接记录

## 本次完成
概述已交付功能及对应提交；未完成时明确当前阶段。

## 接手前先读
requirement.md → design.md → status.md → test-report.md → review.md

## 关键文件
路径 | 本次变化 | 为什么需要

## 可复现验证
环境、准备条件、执行命令、预期与实际结果。

## 剩余工作
事项 | 原因 | 下一步 | 责任人

## 风险与决定
限制、未覆盖场景、已接受风险及决定依据。

## 下次开始
先检查工作区，再读取上述材料；不要覆盖已有改动。
```

3.6 建立更新触发条件与资料核对习惯

项目资料需要维护，否则“项目大脑”会变成一组过期文件。建议把更新动作嵌入交付门槛：修改公开接口时检查上下文与 README，改变技术路线时检查 ADR，新增数据库规则时检查数据库知识和 SQL 案例，发现误报或漏报时补充代表性样例。文档角色提出变化，相关负责人确认事实，当前任务记录说明哪些资料已更新。不要要求每次微小修改都重写全部文件，也不要依赖月底统一整理，因为那时关键决策的背景往往已经丢失。

每周或每个迭代可抽查几个文件，比较文档路径是否存在、命令是否仍可执行、未知项是否得到处理，以及结论是否有来源和日期。发现冲突时优先核对当前代码、测试与已确认决策，不能简单选择更新时间较新的文本。对于已经不适用的知识，明确标注适用版本或移入归档，避免新的 AI 会话误用旧结论。可把“读取这些材料后指出冲突和过期项”作为独立任务，让 AI 辅助检查；最终修正仍需要有依据的事实判断和对应维护者负责。

表 3-2 项目资料更新矩阵

变化触发	需要检查的文件	核对责任
新增模块或接口契约变化	AI_CONTEXT.md、README.md、相关设计	技术负责人
工程规范或权限流程变化	AGENTS.md、AI_RULES.md、AI_WORKFLOW.md	技术负责人及流程负责人
长期架构取舍	architecture/adr 中的记录	架构负责人
数据库版本、语法或规则变化	database 资料、sql-case、对应测试	数据库专家与测试负责人
结束一次工作会话	任务 status.md 与 handoff.md	当前执行者
任务交付	任务全部证据与必要的长期知识	任务负责人

第四章 六种 AI 角色与交接规范

4.1 产品 AI：把业务目标变成可测试需求

产品 AI 负责将业务表达转换为可验收的需求，首先识别用户、使用场景、输入输出和例外情况。例如“支持 GaussDB 审核”可能涉及多个产品形态、不同版本和不同语法范围，不能直接理解为新增一个字符串枚举。产品角色应区分必须确认的事实与可以暂缓的选择，并把每项验收条件写成能够观察的行为。它还需要明确本次是否包含连接数据库、执行计划分析、规则配置界面和历史结果兼容，防止任务在实现过程中不断扩张。

产品 AI 的输入是业务目标、现有产品行为、样例 SQL 和可用数据库资料；输出是 requirement.md 及待确认清单。交接门槛是范围清晰、验收可验证、关键产品信息有依据。无法获得完整信息时，可以先交付受限范围的分析，但必须把限制写进需求。业务负责人保留最终需求判断，AI 负责整理和发现遗漏。使用下方提示词后，应检查它是否提出有实际影响的问题、是否虚构数据库支持程度，以及是否把方案细节提前写成不可更改的需求。

```
你承担本任务的产品分析角色。
先读取 AGENTS.md、.ai/AI_CONTEXT.md、.ai/AI_RULES.md,
以及.ai/tasks/GAUSS-001/中已有的需求与状态。
任务目标: 在 SQL 审核平台中增加目标 GaussDB 产品的静态审核支持。

请检查目标用户、产品名称、部署形态、版本、SQL 范围与已有结果契约。
把已确认事实与未知项分开；缺少资料时不得虚构语法或能力。
输出.ai/tasks/GAUSS-001/requirement.md，包含:
业务目标；范围与不在范围内的内容；输入输出；
异常与不支持行为；可验证验收条件；兼容要求；未决项。
每项验收写出输入、期望行为和证据类型。
在末尾列出架构角色可以开始的内容及仍受未知项限制的内容。
本阶段不修改源码，不自行承诺生产发布。
```

4.2 架构 AI：在既有扩展点中解释方案取舍

架构 AI 要先理解现状，再提出最小必要变化。对于 SQL 审核平台，需要检查数据库适配接口、解析器选择机制、规则调度、结果模型和错误处理，判断新增支持是否能够沿用已有扩展点。方案不能只列出 GaussDBAdapter 等类名，还应说明它们为什么需要、输入输出是什么、哪些已有模块会受影响。如果目标方言存在不支持语法，应设计明确的失败或降级行为，并防止系统把解析失败误报为审核通过。

架构角色的输入包括已确认需求、相关代码、现有测试、数据库资料和架构决策；输出为 design.md 与必要的 ADR。交接门槛是修改边界明确、兼容影响有说明、关键取舍有依据、开发者能够按计划实现。涉及长期依赖或架构方向时，由技术负责人确认。提示词应要求至少比较一个可行替代方案，但不鼓励为了展示复杂度设计过多抽象。架构 AI 还应说明哪些判断必须通过真实产品版本验证，避免把对相近数据库的认识直接迁移为结论。

```
你承担本任务的架构角色。
读取项目入口、上下文、规则，以及 GAUSS-001 的 requirement.md。
阅读实际审核入口、适配器、解析器、规则调度、结果模型与相关测试，
在结论中引用真实路径；找不到的模块明确写为待定位。

输出.ai/tasks/GAUSS-001/design.md：
1. 当前行为与扩展机制；
2. 最小修改方案，列出文件、职责、输入输出与错误处理；
3. 一个可行替代方案及取舍；
4. 已有 MySQL 与 TiDB 行为的兼容检查；
5. 不支持语法、解析失败与未知数据库类型的行为；
6. 验证计划、回退方法、风险与待核实事项。
长期决策写入.ai/architecture/adr/并与任务设计互相引用。
不得默认 GaussDB 等同于 PostgreSQL，也不得自行新增未经确认的依赖。
以"开发前必须满足的门槛"结束，当前阶段不修改生产代码。
```

4.3 开发 AI：把方案拆成可审查的小批次修改

开发 AI 承担实现工作，开始前必须检查当前工作区、已有改动和任务材料。它应该围绕设计中的扩展点制定计划，每个批次包含明确的修改文件与完成条件，例如先加入数据库类型识别，再加入适配器，最后补充代表性规则与测试。代码生成完成后需要阅读差异，检查命名、异常路径和结果契约，不能只依赖生成时的推理。发现方案与代码实际结构不一致时，先记录差异并修正相关设计，而不是悄悄扩大改动范围。

开发角色输入是需求、设计、工作规则和当前代码；输出是代码差异、plan.md、status.md 与自检结果。交接门槛是修改符合设计、基础构建可验证、已知问题清楚记录。开发者应保留用户改动并避免无关重构，尤其不要为了引入一种数据库就同时整理所有模块。若运行环境不可用，要准确记录未验证项并继续处理可做的检查，不能写成“测试通过”。下方提示词将计划、实现和交接连在一起，便于任务跨会话持续推进。

```
你承担本任务的开发角色。
先读 AGENTS.md、AI_CONTEXT.md、AI_RULES.md、AI_WORKFLOW.md，
再读 GAUSS-001 的 requirement.md、design.md、plan.md、status.md。
检查当前工作区并保留已有修改；以真实代码结构为准。

先更新 plan.md：列出修改文件、实施顺序、每一步完成条件与验证方式。
在已授权范围内实施最小必要变化，分批检查代码差异。
生产代码使用现有包名与目录；教学项目路径为：
src/main/java/com/example/audit/
测试路径为 src/test/java/com/example/audit/
SQL 样例路径为 src/test/resources/sql/

新增依赖、契约变化或扩大范围时，按既有团队规则处理并记录原因。
运行适合本次变更的检查；如无法运行，明确写出环境原因与未验证项。
更新 status.md 与 handoff.md，列出改动、结果和剩余工作。
不得虚构命令结果，不连接生产数据库执行提交的 SQL。
```

4.4 测试 AI：从验收条件推导验证，而不是复制实现

测试 AI 需要独立于实现思路检查业务要求，防止开发与测试共同遗漏同一个问题。测试设计应覆盖正常输入、边界语法、未知数据库类型、不支持语法、解析异常和已有数据库回归，必要时还要验证并发、大小限制和结果顺序。对于规则审核，要同时提供命中与不命中案例，明确期望风险级

别和解释。只检查返回对象不为空，不能证明规则正确；只依据代码当前行为生成断言，也可能把错误行为固化为测试标准。

测试角色输入是需求、设计、代码差异、SQL 样例和可用环境，输出为可运行测试与 test-report.md。交接门槛是验收条件有相应证据，失败有处理记录，环境限制与未覆盖场景得到明确说明。单元测试证明本地逻辑，集成验证检查模块交互，真实数据库验证则只在需求需要、环境可用且已授权时开展。三类证据不可混淆。测试 AI 应主动寻找能够推翻实现假设的样例，并记录样例来源与脱敏情况，便于之后回归。

```
你承担本任务的测试角色。
读取 GAUSS-001 需求、设计、代码差异和现有测试，不仅依据实现推导期望。
为每项验收条件建立验证对应关系，覆盖：
正常 SQL、命中与不命中、未知数据库、不支持语法、解析异常、
已有 MySQL 与 TiDB 回归，以及与任务有关的资源边界。

将测试放入 src/test/java/com/example/audit/，
将脱敏 SQL 样例放入 src/test/resources/sql/，遵循现有工程习惯。
记录样例目的与预期，避免仅验证"对象不为空"。

实际运行项目规定的检查。输出 test-report.md：
环境与版本；准备条件；执行命令；用例与验收映射；
实际结果；失败与修复记录；未运行项；剩余覆盖限制。
单元通过不得写成真实数据库验证通过。
遇到失败先定位并报告可复现证据，不自行改变业务验收标准。
```

4.5 Review AI：围绕缺陷与证据进行独立审查

Review AI 的职责是发现可能影响正确性、兼容性、维护或权限边界的问题，而不是对代码风格发表泛泛建议。它应从需求、设计、最终差异和测试报告出发，检查数据库选择是否错误、解析失败是否被吞掉、规则级别是否一致、共享状态是否安全以及已有行为是否改变。每条发现需要指出受影响路径、触发条件和实际后果；不能证明的问题可以列为待验证事项，但不能编造复现过程。审查结论也不能因为使用了另一个模型就被视为天然可靠。

Review 角色输入是最终代码差异与交付证据，输出为 review.md 和可处理的问题清单。交接门槛是阻断问题解决或按团队流程作出明确决定，剩余风险可追踪。审查者应优先指出影响业务的缺陷，避免用大量低价值格式意见淹没关键问题。修复后重新检查受影响部分及相关测试，不需要无理由重复全部检查。若审查没有发现问题，应注明审查范围、证据和未覆盖限制，不能把“未发现”写成不存在任何缺陷。

```
你承担本任务的独立代码审查角色。
读取 GAUSS-001 需求、设计、最终代码差异、test-report.md 与现有约束。
围绕正确性、数据库选择、错误处理、契约兼容、共享状态、
资源限制、敏感数据和测试缺口检查，不只关注格式。

输出 review.md，包含审查范围、依据与发现。
每个发现写出：严重程度；文件与定位；触发条件；
可说明的后果；依据；修复建议；需要补充的验证。
不能复现时明确标为待验证，不虚构运行结果。
无发现时写清覆盖范围和限制。

检查 MySQL 与 TiDB 是否因本次扩展受影响，
检查未知类型或解析失败是否被当作安全通过。
阻断问题关闭后记录处理证据；未决定风险交由对应负责人。
本阶段先报告问题，修改代码须在已授权的修复范围内进行。
```

4.6 文档 AI：把本次发现提炼为下次可用知识

文档 AI 需要区分一次任务的过程证据与能够长期复用的结论。任务目录保存本次需求、差异、测试与决策；知识库只保留有稳定价值的材料，例如目标数据库特定版本的语法限制、规则误报原因和适配接口的使用方式。直接把聊天记录复制成文档，往往会留下重复推测、过期方案和没有依据的说法。文档角色应从最终代码和验证结果出发，删去已放弃方案，给结论标注适用范围、来源和核对日期。

文档角色输入是最终交付物、审查结果与已确认决策，输出为 handoff.md 及必要的总览、ADR 和知识更新。交接门槛是接手者能够找到相关文件、复现验证并理解剩余限制。维护者需要核对命令和路径是否真实存在，对未验证信息保持清晰标签。任务完成不代表所有知识都已成熟；存在环境限制时，可以记录“待验证假设”，但不能放入数据库事实清单。这样既保留有价值的探索，也降低下一次 AI 读取资料时误用结论的风险。

```
你承担本任务的文档与交接角色。
读取 GAUSS-001 最终需求、设计、状态、测试报告、审查及代码差异。
以最终实现与真实验证为依据，整理 handoff.md：
本次完成；关键路径；验证方法与实际结果；
剩余风险、未验证项；接手者下一步。

检查 AI_CONTEXT.md 是否需更新；工程规则变化时检查 AI_RULES.md；
长期架构取舍放入.ai/architecture/adr/；
可复用事实放入.ai/knowledge/或.ai/database/；
脱敏案例放入.ai/sql-case/，并链接对应测试。

不要复制完整聊天记录，不保留已经放弃的方案作为现状。
每条数据库结论注明产品、版本、来源与核对日期。
区分事实、假设和待验证项。
列出更新文件与理由，交由相关维护者核对后随任务交付。
```

4.7 用 RACI 与交接包避免角色空转

六种角色是工作视角，不一定对应六个模型或六位员工。一个三人团队可以由同一位开发者组织 AI 完成需求整理、实现和文档，由另一位同事审查；较大团队则可以按专业分工拆开。关键是每一项成果都有执行责任与最终确认责任，角色之间通过具体文件交接。RACI 表中的 R 表示执行，A 表示最终负责，C 表示提供专业意见，I 表示知情。AI 角色可协助执行与咨询，但最终负责应由团队中拥有相应职责的人承担。

实际使用时，为每次任务写一个交接包：先列本阶段输入路径，再列输出文件、关键决定、未决事项和下一阶段门槛。接收角色首先检查材料是否完整，然后开展自己的工作，不把上一角色的自我评价当成事实。图 4-1 说明六种角色通过同一个任务目录交接，测试与审查可以把发现返回设计或开发。若角色切换没有新的检查视角，只是重复赞同上一份输出，就不值得增加复杂度；应调整提示词，让每个角色明确自己的独立问题与证据要求。



图 4-1 六种角色通过任务目录交接

表 4-1 人类责任与 AI 协助的 RACI 示例

工作项	业务负责人	技术负责人	任务开发者	测试负责人	审查者	AI 角色
需求与验收	A	C	R	C	I	产品 AI 协助 R
架构与依赖决策	C	A	R	C	C	架构 AI 协助 R
代码实现	I	A	R	C	C	开发 AI 协助 R
测试执行与证据	C	A	R	R	C	测试 AI 协助 R
代码审查	I	A	C	C	R	Review AI 协助 R
知识更新与交接	C	A	R	C	C	文档 AI 协助 R
正式发布与业务验收	A	R	C	C	C	提供材料与检查建议

```
# 阶段交接包

任务编号：GAUSS-001
发送角色：待填写
接收角色：待填写

## 输入已核对
文件路径 | 核对时间或版本 | 重要结论
```

本阶段输出
文件路径 | 作用 | 完成状态

关键决定
决定 | 依据 | 确认责任人

未决事项
事项 | 影响范围 | 可继续的工作 | 必须满足的门槛

接收者检查
- 输入是否完整、可访问;
- 关键事实是否有证据;
- 本阶段门槛是否真的满足;
- 下阶段修改范围与验证目标是否明确。

第五章 需求澄清与架构设计

本章使用 SQL 审核平台增加 GaussDB 适配能力作为贯穿案例。技术栈暂定为 Java 17、Spring Boot 与 Maven，这是便于讲解的教学假定，不代表对读者现有项目的判断。我们从一句模糊需求出发，依次写出产品需求、能力边界和架构决策。这里的“支持”指在明确数据库目标下识别 SQL、应用经过验证的规则、返回可解释结果，不包含直接执行用户提交的 SQL。全部示例都是设计与实现方法示范，尚未连接读者的真实项目或数据库，不能视为已经完成上线验收。

5.1 步骤一：确认目标产品、部署、版本与兼容模式

接到“帮平台支持 GaussDB”时，先不要让开发 AI 创建类。打开 .ai/tasks/GAUSS-001/requirement.md，把问题拆成四个必须确认的字段：准确产品名称、集中式或分布式部署、实例内核版本、数据库兼容模式。华为云官方资料分别列出集中式与分布式实例，兼容性说明也指出不同兼容模式存在差异。[S_GAUSS_INSTANCE][S_GAUSS_COMPAT] 因此，名称相似、连接协议相近或某条查询能够运行，都不能作为全面兼容的证明。GaussDB、openGauss、PostgreSQL，以及带其他后缀的产品名称，应作为不同的验证目标登记。

实际操作时，由产品负责人提供控制台脱敏截图或实例信息清单，由数据库负责人核对版本与模式，再将证据位置写入需求文件。无法取得信息时，把字段写成“待确认”，同时允许 AI 继续梳理现有审核入口、结果结构和扩展点。不能让 AI 用常见版本补齐空白，因为后续语法解析、元数据查询和集成环境都会依赖这些字段。目标登记完成后，用同一份登记表命名测试目录，例如 gaussdb/target-a，避免目录名只写 gaussdb 而把多个模式的样本混在一起。

表 5-1 目标数据库确认清单

字段	填写方式	未确认时的处理
产品	云数据库 GaussDB；记录控制台产品全名	保留待确认，不套用 openGauss 文档
部署	集中式或分布式；记录相关组件范围	不承诺执行计划和分布式规则
版本	实例实际内核版本与补丁信息	只做接口设计，不宣称语法支持
兼容模式	记录实际模式及影响行为的参数	不给出跨模式结论
证据	脱敏截图、负责人、采集日期	状态保持需求澄清中

```
# GAUSS-001 目标登记
产品全名：待数据库负责人确认
部署形态：待确认
内核版本：待确认
兼容模式：待确认
相关兼容参数：待确认
实例信息证据：待补充，禁止保存口令与连接密钥
确认人：待指定
确认日期：待填写
教学目标：先完成离线审核接口与失败关闭行为
生产支持声明：尚未形成；不得将本登记用于承诺全量兼容
```

5.2 步骤二：把业务意图写成完整 PRD

需求文件的重点是说明谁遇到了什么问题，以及什么结果才算解决。产品 AI 应先阅读现有用户流程，例如开发者上传 SQL 后，审核人员查看风险项，管理员维护规则。然后将新增目标拆成可观察行为：用户能选择经过确认的数据库目标；系统明确报告是否完成解析；不能识别的语法返回“不支持”；已有 MySQL、TiDB 目标维持约定的结果。每条行为要对应输入样本与预期结果，不能只有“体验友好”“性能良好”这样的形容词，否则测试 AI 没有可重复的判定依据。

创建 requirement.md 后，先填写背景、范围、非目标、验收条件和待决问题，再让产品 AI 补充异常路径。输入为空、目标模式缺失、SQL 超限、解析超时、规则无法求值，都是用户会遇到的业务场景。性能指标应由现有平台基线和业务需求给出，暂时未知就写明待测，不凭空规定一个漂亮数字。需求评审时请人工逐条确认条件，特别是“没有发现风险”和“已经完整审核”之间的区别。没有发现风险也可能只是没有运行规则，界面和接口必须让使用者看见这一差异。

```
# GAUSS-001 产品需求文档
## 1. 背景与用户
SQL 提交者需要知道指定 GaussDB 目标下的审核结论。
审核人员需要看到规则编号、风险说明、证据位置和未覆盖范围。
## 2. 目标数据库
引用本文件的目标登记；所有待确认项在支持声明前关闭。
## 3. 本期范围
离线审核入口；显式目标选择；结果状态；失败关闭；目标语料测试。
规则清单：由能力矩阵确认，不将未验证规则写成已支持。
## 4. 非目标
不执行提交 SQL；不自动修复并回写数据库；不承诺所有 GaussDB 版本。
不以 openGauss/PostgreSQL 测试代替目标 GaussDB 验证。
## 5. 用户流程
选择目标 → 提交 SQL → 查看状态 → 查看规则结果 → 人工处理。
## 6. 输入约束
产品、部署、版本、模式必须完整；限制 SQL 字节数与语句数量。
编码、超时、批量语句处理方式由设计文档补充并经评审确认。
## 7. 验收条件
AC-01：目标不完整时返回明确错误，不产生通过结论。
AC-02：解析器未提供或语法不支持时，返回 UNSUPPORTED。
AC-03：审核结果包含状态、规则编号、说明和目标信息。
AC-04：MySQL/TiDB 已有固定语料的结果没有未经确认的变化。
AC-05：支持声明引用目标实例的集成验证报告。
## 8. 异常与观测
分别记录输入无效、解析失败、不支持、规则异常和超时。
日志只记录任务标识与脱敏指纹，不保存密钥或完整敏感 SQL。
## 9. 待决问题
目标版本与模式；解析器来源及许可；规则范围；性能目标；发布责任人。
## 10. 评审记录
产品负责人：待指定；数据库负责人：待指定；状态：草案。
```

5.3 步骤三：构建能力矩阵，限制支持声明的范围

能力矩阵把一个大词“支持”拆成语法、语义、规则、元数据、执行计划和环境验证等不同维度。某个解析器能够生成语法树，只证明它处理了该样本的结构；不能自动证明规则理解了数据库的真实行为。相反，一个规则在人工构造的节点上返回正确，也不能证明解析器生成了正确节点。让架构 AI 为每一项填写当前证据、验证目标与缺口，保存到 .ai/database/gaussdb-capabilities.md，并在 design.md 中引用该文件。矩阵还应包含“未覆盖”，这样后续扩展才有清晰入口。

初学者可以先划定很小的规则范围，例如对已经可靠解析的单条语句检查明确列出的风险，不同时开展存储过程、方言函数、复杂 DDL 和分布式性能诊断。实际模式相关的 SQL 行为还可能受兼容参数影响，官方参数文档应按目标版本逐项核对。[S_GAUSS_PARAMS] 表中“待验证”不能在演示时改成“支持”，也不能因为测试数量较多就省略版本条件。每增加一项能力，都应添加正向样本、反向样本和不能判定的样本，并由数据库负责人确认预期结果。

表 5-2 教学初始能力矩阵

能力	当前状态	必要证据	失败行为
目标识别	接口示例已设计	目标登记与输入校验测试	INVALID_INPUT
真实方言解析	尚未实现	解析器来源、许可、目标语料	UNSUPPORTED
风险规则	尚未实现	规则规范、预期结果、误报漏报评估	不产生通过
元数据读取	本期可先排除	只读权限、查询许可清单、超时	结果标记信息不足
执行计划分析	本期排除	单独风险评审与目标环境验证	不调用数据库
真实目标集成	尚未运行	实际产品/版本/模式记录和报告	不能发布支持声明

5.4 步骤四：写架构方案和 ADR，记录取舍

设计文档描述本次需求将如何进入现有系统，ADR 则记录为什么选择某种方案。先让架构 AI 定位当前数据库类型枚举、解析器工厂、规则入口、响应对象和测试目录，把事实与建议分开。若项目已经采用插件架构，就优先扩展既有接口；若没有，先比较小范围适配与整体重构的成本。不能因新需求引入一个新数据库，就顺便改造所有模块。为本次关键决定创建 .ai/architecture/adr/0001-gaussdb-offline-adapter.md，说明选择理由、代价、替代方案及何时重新评估。

本案例建议的决定是离线审核优先、适配器隔离、未知能力失败关闭。这是教学方案，应由项目负责人根据真实业务评审。设计中要写出数据怎样流动：请求先校验目标，注册表找到适配器，适配器调用经过验证的解析器，规则引擎接收结构化节点，结果装配器输出状态与发现项。每个边界都有错误出口，不能只有一路成功的流程图。ADR 还要记录现实代价，例如离线模式缺少表统计信息，因此不能宣称能判断所有执行性能问题，这使支持范围可解释、可维护。

```
# ADR-0001: GaussDB 采用隔离适配器与离线审核
状态: 提议; 待架构负责人评审
关联任务: GAUSS-001
## 背景
目标产品、版本、部署与模式存在差异，需要隔离方言能力。
## 决定
第一阶段只离线处理输入 SQL，不执行用户提交语句。
通过 AuditEngine 接口注册目标适配器；无真实解析能力时返回 UNSUPPORTED。
审核状态与规则风险分别表示；信息不足不转成通过。
## 备选方案
直接复用 PostgreSQL 解析器: 成本低，但目标兼容性没有证据。
为全部数据库重写引擎: 范围大，与本次验收目标不匹配。
## 后果
短期覆盖范围有限；需要语料与目标实例验证后逐步开放。
优点是已有数据库适配器边界清晰，失败结果可解释。
## 重新评估条件
目标确认；解析器验证完成；需引入元数据或执行计划时单独评审。
## 证据
requirement.md; gaussdb-capabilities.md; 后续 test-report.md。
```

5.5 步骤五：评审插件边界与修改清单

架构评审不是让多个 AI 对方案表示赞同，而是把风险逐项落实到接口和证据。重点检查适配器能否带上完整目标信息，解析失败是否与审核通过分离，以及新增目标是否会改变现有 MySQL、TiDB 的路由。将计划修改的文件写成明确清单，例如 AuditEngine.java、GaussDbAdapter.java、对应测试和任务文档。对于还没有看到的现有项目文件，应使用“待定位”的标记，不能编造某个 ParserFactory 已存在。只有检查仓库后，清单里的修改才能从建议变为确定工作。

阅读下图时，从左侧输入开始，逐个检查箭头旁的契约。目标信息不完整时应停在校验层；找不到相应适配器时应返回不支持；解析器拒绝语法时不能进入规则判断；规则运行中发生异常应保留未完成状态。图中的只读元数据是未来可选能力，不等于授权执行任意查询。评审结束后把结论写入 design.md 的决策区，列出通过项、待补证据、责任人和下一步。这样开发 AI 能遵循已确定的边界，测试 AI 也能从同一方案生成契约测试。



图 5-1 SQL 审核适配器架构：每层保留明确失败出口

第六章 任务拆解与适配器实现

这一章把已评审的需求变成能够执行的小任务。任务卡不仅要告诉 AI 写什么，也要说明先做什么、依赖什么、如何判断完成。教学示例只提供接口、目标校验和保持不支持状态的适配器骨架，故意不假装存在一个完整 GaussDB 解析器。读者复制示例后得到的是安全边界与扩展入口，仍需接入经过验证的解析器、规则和目标实例测试，才能向用户承诺具体能力。

6.1 步骤一：按证据拆分任务，而不是按文件平均分工

打开 .ai/tasks/GAUSS-001/plan.md，把工作拆成有前置条件、有产出和有退出标准的单元。先完成目标登记与需求评审，再确认接口契约，然后准备语料、实现目标校验和适配器骨架。解析器研究可以与语料收集并行，但规则实现必须等待解析节点语义得到确认。不要把“写三个类”作为计划终点，因为三个类即使存在，也可能完全没有满足业务要求。每一项工作最好能在较短周期内形成可审查的文件、差异或报告，失败时也能独立说明原因。

AI 可以帮助生成依赖图，但任务负责人必须检查依赖是否真实。比如测试 AI 能提前设计目标缺失的预期结果，却不能在解析器尚不存在时报告方言解析测试通过。为每项任务分配一个负责角色与一个审核角色，避免多个 AI 同时修改同一个公共接口。并行工作前冻结结果模型与规则编号格式，把冻结位置写入任务卡。任务完成后提交输出和证据，负责人确认后再推进依赖任务。这样既能利用并行能力，也不让并行产生互相矛盾的代码。

表 6-1 GAUSS-001 子任务与退出标准

任务	依赖	输出	退出标准
A 目标与需求	无	requirement.md	人工确认目标与范围
B 接口与边界	A	design.md、ADR	状态和失败行为明确
C 语料基线	A	sql/gaussdb/target-a	预期结果经人工核对
D 安全骨架	B	接口、适配器、测试	未支持情况不会放行
E 解析与规则	B、C、解析器证据	真实实现及能力矩阵	目标语料逐项有证据
F 回归与集成	D、E、目标环境	test-report.md	支持条件与缺口明确
G 审查与交接	F	review.md、handoff.md	问题关闭或明确阻塞发布

```
# GAUSS-001 实施计划
当前阶段：D 安全骨架；A/B/C 在真实项目中仍需核实
接口基线：src/main/java/com/example/audit/AuditEngine.java
适配器：src/main/java/com/example/audit/GaussDbAdapter.java
测试：src/test/java/com/example/audit/GaussDbAdapterTest.java
语料：src/test/resources/sql/gaussdb/target-a/
修改约束：不改变已有 MySQL/TiDB 路由，不引入 SQL 执行能力。
完成标准：状态契约、输入校验、UNSUPPORTED 行为有测试证据。
剩余工作：真实解析器、规则、目标环境集成、性能基线、发布验证。
每次交付：说明本次差异、运行检查、失败项和下一步。
```

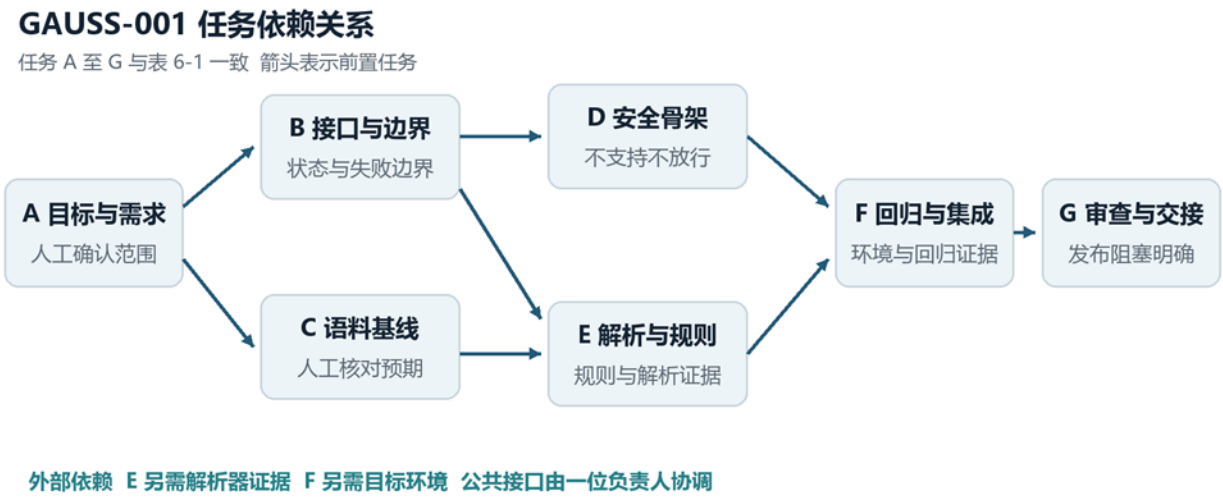


图 6-1 GAUSS-001 任务依赖图：语料准备与实现相互校验

6.2 步骤二：先定义审核状态和核心接口

在 src/main/java/com/example/audit 下创建 AuditEngine.java。下面的示例使用 Java 17 的记录类型，读者若使用旧版 Java，应由开发人员调整为普通类，不能直接降低编译要求后忽略错误。接口将目标信息、处理状态和风险发现分开表示。INVALID_INPUT 说明请求不满足输入条件；UNSUPPORTED 说明能力尚未覆盖；REVIEWED 说明约定范围内的处理完成。是否可以进入后续人工流程，还要检查阻断级发现项。生产项目如果存在更多状态，应保留既有语义并补充迁移测试。

复制接口之后，先审查 allowed 方法的条件，而不是马上实现风险规则。它只允许处理完成且没有阻断项的结果，防止空发现列表被误认为通过。这个设计仍是最小示例，实际系统还要考虑被跳过的规则、信息不足、批量语句、规则版本与解析证据等字段。若部分规则未运行，应将状态保持未完成，或用清晰的覆盖状态阻止自动放行。AI 输出接口时必须解释每个字段由哪一层填写，否则相同的状态名称容易在不同模块代表不同含义。

```
// 文件: src/main/java/com/example/audit/AuditEngine.java
package com.example.audit;

import java.util.List;
import java.util.Objects;

public interface AuditEngine {
    AuditResult audit(String sql, Target target);

    record Target(String product, String deployment,
                  String version, String mode) {
        public boolean complete() {
            return present(product) && present(deployment)
                && present(version) && present(mode);
        }
        private static boolean present(String value) {
            return value != null && !value.isBlank();
        }
    }

    enum Status { INVALID_INPUT, UNSUPPORTED, REVIEWED }
    enum Severity { BLOCK, WARN }
```

```
    record Finding(String ruleId, Severity severity, String message) {
        public Finding {
            Objects.requireNonNull(ruleId);
            Objects.requireNonNull(severity);
            Objects.requireNonNull(message);
        }
    }

    record AuditResult(Status status, List<Finding> findings,
                       String detail, Target target) {
        public AuditResult {
            Objects.requireNonNull(status);
            findings = List.copyOf(findings);
            Objects.requireNonNull(detail);
        }
        public boolean allowed() {
            return status == Status.REVIEWED
                && findings.stream().noneMatch(
                    item -> item.severity() == Severity.BLOCK);
        }
    }
}
```

6.3 步骤三：实现诚实返回 UNSUPPORTED 的适配器骨架

创建 GaussDbAdapter.java，把请求完整性和资源边界先落地。示例只检查目标、输入空白和 SQL 字节长度，不进行语法推断。任何完整请求都返回 UNSUPPORTED，因为尚未接入真实解析器。这样的骨架仍然有价值：它让前端、路由、状态显示、日志和测试可以围绕一个稳定契约工作，同时不会误导用户以为风险检查已经完成。长度上限只是教学参数，应在真实项目中配置化，并依据业务基线与平台约束确定。不要把它作为 GaussDB 官方限制引用。

不能用“包含 SELECT 就通过”替代解析，也不能把分号拆分或正则匹配称为完整方言支持。字符串可能含注释、转义、嵌套结构和多个语句，错误的结构识别会使规则漏掉风险。接入解析器时应保持这个失败出口：只有目标明确、语法成功解析、必需规则全部完成后才能产生 REVIEWED。若解析器没有提供目标模式保证，先保持不支持并收集证据。示例没有连接数据库、创建 Statement 或执行 SQL；未来引入元数据能力时，需要单独设置查询许可清单、超时和权限。

```
// 文件: src/main/java/com/example/audit/GaussDbAdapter.java
package com.example.audit;

import java.nio.charset.StandardCharsets;
import java.util.List;

public final class GaussDbAdapter implements AuditEngine {
    // 教学限制；生产项目应配置化并经过容量测试。
    private static final int MAX_SQL_BYTES = 64 * 1024;

    @Override
    public AuditResult audit(String sql, Target target) {
        if (target == null || !target.complete()) {
            return result(Status.INVALID_INPUT,
                "目标产品、部署、版本、兼容模式必须完整", target);
        }
        if (!"GaussDB".equals(target.product())) {
```

```
            return result(Status.UNSUPPORTED,
                "本适配器仅接收显式 GaussDB 目标", target);
        }
        if (sql == null || sql.isBlank()) {
            return result(Status.INVALID_INPUT, "SQL 不能为空", target);
        }
        if (sql.getBytes(StandardCharsets.UTF_8).length > MAX_SQL_BYTES) {
            return result(Status.INVALID_INPUT, "SQL 超过教学大小限制", target);
        }
        // 此处没有真实方言解析器，因此绝不能返回 REVIEWED。
        return result(Status.UNSUPPORTED,
            "教学骨架：尚未实现目标方言解析与规则审核", target);
    }

    private AuditResult result(Status status, String detail, Target target) {
        return new AuditResult(status, List.of(), detail, target);
    }
}
```

6.4 步骤四：逐项接入解析器与规则，控制修改范围

真实实现从验证解析器开始。记录解析器名称、来源、版本、许可、支持声明和适用目标，再用准备好的 SQL 语料检查解析树。先人工核对关键节点，例如语句类型、表名、过滤条件和函数调用，再让规则消费这些节点。不要仅判断“没有抛异常”，因为某些解析器可能接受文本却丢失重要结构。每一条规则应有规范文件，说明适用语句、风险含义、所需信息、严重级别、例外与不能判定时的行为，保存到 .ai/knowledge 或现有规则规范目录。

每次只提交一个可审查的能力增量，例如添加一个可靠语句类型及其对应规则，而不是一次覆盖全部方言。开发 AI 的输入应包括 design.md、能力矩阵、准确修改清单和语料位置，输出应包含差异说明与验证结果。修改公共接口时先分析已有适配器调用方，必要时采用兼容扩展，避免迫使所有目标同时迁移。对于只有真实数据库才能确认的语义，把单元测试结果和待集成验证分开写。代码完成不等于任务完成，能力矩阵只有在相应证据形成后才更新为已验证。

```
请执行 GAUSS-001 的一个最小能力增量。
先读取 requirement.md、design.md、plan.md 和当前能力矩阵。
定位真实解析器入口，报告其来源、版本与目标能力证据。
只实现本任务明确列出的语句类型和规则；未知语法返回 UNSUPPORTED。
不执行提交 SQL，不修改无关数据库适配器。
完成后报告：修改文件、行为变化、运行检查、证据路径、未验证项。
若关键前置证据缺失，先完成可独立交付部分，并记录缺口。
```

6.5 步骤五：维护 status.md，让任务可恢复

每次结束工作前，更新 .ai/tasks/GAUSS-001/status.md。状态文件要像交班记录一样可读，说明当前处在哪个阶段、已经完成哪些实际产物、哪些检查真正运行、哪些结论仍待验证。使用“设计完成”“骨架检查完成”“目标集成待运行”等精确词语，避免一句“基本完成”掩盖重要缺口。记录最近的提交标识或文件版本，使接手者能找到同一份代码；如果工作尚未提交，也应说明当前差异与保存位置。不要把聊天中的最后一句话当作任务唯一状态。

恢复任务时，AI 首先读取状态文件，再核对需求和当前工作区。如果文件说明某项已通过，但没有报告或当前代码已经变化，应重新确认相应证据，而不是盲目继承旧结论。状态文件无需复制所有日志，保留摘要和证据链接即可。真实项目中的阻塞要记录所缺的信息、影响和负责人，例如缺

少目标实例导致无法验证兼容模式，不应写成含糊的“环境问题”。教学示例应一直保留“解析与规则尚未实现”的状态，直到读者完成这些工作，避免把教材中的演示当成项目事实。

```
# GAUSS-001 当前状态
更新时间：填写实际日期与时区
当前阶段：安全骨架示范
已形成：目标登记模板、PRD、ADR、接口、UNSUPPORTED 适配器示例。
实际验证：本教材不代表读者项目已经运行测试。
尚未完成：产品确认、真实解析、规则、目标实例集成、发布验收。
风险：空发现列表不得转换成通过；旧数据库回归仍需执行。
证据：引用实际报告路径与提交标识；未运行项明确写“未运行”。
下一步：确认目标 → 选择并验证解析器 → 准备目标语料。
责任人：待指定；不得把 AI 角色名称当作最终业务责任人。
```

第七章 测试验证与代码审查

AI 生成的代码要靠独立证据建立信任，测试数量本身并不能说明系统可靠。本章从语料、契约、回归和目标环境验证四个层次构建检查。读者应优先覆盖真正影响业务结果的边界，例如未知语法会不会被放行、已有数据库结果是否变化、错误报告能否帮助定位问题。对低影响文案或简单目录创建，不需要额外堆积镜像实现的测试；对决定审核通过的核心逻辑，则需要能揭示错误的检查与人工确认。

7.1 步骤一：建立 SQL 语料和人工确认的预期结果

在 src/test/resources/sql/gaussdb/target-a 下放置 SQL 样本与预期文件，target-a 的说明必须引用目标登记。语料至少包括正常语句、确定触发风险的语句、当前不支持的语句和容易误判的边界输入。为每条样本分配稳定编号，记录预期状态、预期规则、理由、确认人和资料来源。不要只收集能够通过的 SQL，否则测试集会奖励一个把所有输入都判为安全的错误实现。真实业务样本需先脱敏，去掉用户数据、口令和内部对象信息，同时尽量保留导致问题的结构。

预期结果应由规则规范和数据库事实形成，不能让同一个 AI 根据当前实现反推答案。下面的 DELETE 示例只用于展示风险规则设计，不代表 GaussDB 特有语法结论；骨架阶段的预期仍是 UNSUPPORTED。等真实解析器与规则完成后，才按已确认规则把对应样本更新为 REVIEWED 并带阻断项。新增一个生产故障样本时，应先证明旧版本会表现出错误，再验证修复改变了目标行为，这能避免语料库变成只保存成功案例的展示目录。

```
-- 文件: src/test/resources/sql/gaussdb/target-a/C001.sql
-- 教学风险样本；本阶段尚未实现方言解析。
DELETE FROM demo_orders;
```

```
{
  "caseId": "C001",
  "targetRef": ".ai/tasks/GAUSS-001/requirement.md",
  "expectedStatus": "UNSUPPORTED",
  "expectedRules": [],
  "reason": "骨架尚无真实解析器，不能给出完整审核结论",
  "futureRule": "GAUSS-DML-001 (待规范评审) ",
  "reviewer": "待指定",
  "evidence": "教学样本；未在目标实例运行"
}
```

表 7-1 必须区分的语料类别

类别	目的	最小示例	主要判定
正常	避免全部阻断	规则允许的普通语句	完成状态与无阻断项
风险	防止漏报	已确认应阻断的无条件修改	规则编号与严重级别
不支持	防止能力外放行	未覆盖的语句类型	UNSUPPORTED 且不允许
边界	暴露错误处理	空输入、超限、目标缺失	INVALID_INPUT 且可解释
歧义	防止字符串误匹配	注释、字面量内含关键字	结构正确或明确不支持

7.2 步骤二：选择最小但有效的单元和契约测试

单元测试检查局部行为，契约测试检查调用双方约定是否一致。对于前章的骨架，最重要的单元测试是完整请求仍返回不支持、缺失目标返回输入无效，以及任何这些结果都不能被 allowed 放行。这比检查某个私有方法被调用一次更有业务价值。契约测试还要覆盖结果被接口序列化后，前端和其他服务是否仍能看见 UNSUPPORTED；若 UI 只看 findings 是否为空，就会把服务端正确结果显示成通过，需要在边界上直接发现这个错误。

以下测试使用 JUnit 5，真实项目应采用已有 Maven 测试配置和版本，不要为复制示例升级整个构建系统。示例目标字符串是测试夹具，只用于通过完整性校验，不代表任何真实 GaussDB 模式。执行 mvn test 后保存真实结果、运行日期和提交标识；测试无法运行时，报告缺少依赖或环境的事实，不把示例代码存在写成通过。接入真实规则后增加能区分正确与错误实现的样本，例如把风险输入换成正常输入，预期应发生合理变化，而不是所有测试都只检查非空。

```
// 文件: src/test/java/com/example/audit/GaussDbAdapterTest.java
package com.example.audit;

import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

class GaussDbAdapterTest {
    private final AuditEngine engine = new GaussDbAdapter();
    private final AuditEngine.Target fixture = new AuditEngine.Target(
        "GaussDB", "fixture-deployment", "fixture-version", "fixture-mode");

    @Test
    void completeRequestRemainsUnsupportedWithoutParser() {
        var result = engine.audit("SELECT 1", fixture);
        assertEquals(AuditEngine.Status.UNSUPPORTED, result.status());
        assertFalse(result.allowed());
    }

    @Test
    void missingTargetIsInvalidAndCannotPass() {
        var result = engine.audit("SELECT 1", null);
        assertEquals(AuditEngine.Status.INVALID_INPUT, result.status());
        assertFalse(result.allowed());
    }

    @Test
    void blankSqlCannotPass() {
        var result = engine.audit("   ", fixture);
        assertEquals(AuditEngine.Status.INVALID_INPUT, result.status());
        assertFalse(result.allowed());
    }
}
```



图 7-1 从局部行为到目标环境的验证层次

7.3 步骤三：分别执行旧目标回归与真实数据库集成

新增 GaussDB 适配器后，MySQL 与 TiDB 回归重点检查已有行为是否发生未经确认的变化。冻结一份代表性语料和预期结果，比较状态、规则编号、严重级别以及核心说明，而不是只比较文本格式。公共枚举、路由、响应模型、依赖升级和错误处理都可能影响旧目标，因此测试范围要依据实际差异选择。若只有新增独立类，检查可相对集中；若改动公共解析流程，就必须扩大回归。发生变化时先判断是修复、兼容变更还是回归缺陷，再更新预期。

真实数据库集成只在得到授权的隔离测试环境进行，并记录实际产品、部署、版本、模式、驱动和参数。离线审核测试不需要执行用户 SQL；需要核对数据库语法或元数据时，应由测试方案限定样本、账户权限和清理方式。只读账户也不能被视为可以无限执行查询，因为查询仍可能占用资源。不要用 openGauss 容器或 PostgreSQL 实例替代 GaussDB 验证后宣称通过，它们可用于部分通用逻辑实验，但报告必须写清楚环境差异。未取得真实环境时，把集成项标为未运行，并阻止相应支持声明。

表 7-2 各层检查能够证明什么

层次	能够证明	不能单独证明
单元	状态、局部规则、边界分支	真实数据库语法与行为
契约	接口和调用方理解一致	所有方言能力完整
旧目标回归	固定样本无非预期变化	所有历史场景无缺陷
目标集成	指定实例条件下样本行为	跨版本、跨模式全面兼容
性能验证	指定环境与负载下指标	其他硬件与规模的性能

7.4 步骤四：分开评估误报、漏报和未覆盖

审核系统需要同时面对误报与漏报。误报是正常样本被判为风险，会增加人工处理成本；漏报是风险样本没有被识别，可能让危险变更继续流转；未覆盖则是系统明确不能判断，应该保留不同状态。不能把未覆盖样本从统计里悄悄删掉，也不能把所有阻断都计作规则识别成功。为已标注且在声明范围内的样本建立对照表，统计真正的风险命中、正常通过、误报和漏报，并另外报告不支持比例。统计样本数量和选择方法必须随结果一起呈现。

测试报告还应记录重复性：使用什么提交、什么语料版本、什么目标配置以及哪些命令。对于规则漏报，保留最小反例与修复后的回归样本；对于误报，记录允许场景和例外条件是否经过业务负责人确认。小样本上零漏报不能推导总体零风险，报告应直接写“在本次样本中未发现”，避免绝对承诺。性能也要明确测量起止点，例如只测解析与规则还是包含网络请求，记录超时比例和失败类型，不能把失败请求排除后只展示成功请求的漂亮时延。

```
# GAUSS-001 测试报告
报告状态: 待真实执行; 教材示例没有生产验收结论
代码提交: 待填写; 语料版本: 待填写
目标: 产品/部署/内核版本/模式/参数, 逐项填写
环境: JDK、构建工具、驱动、机器资源、隔离方式
## 实际执行
检查名称 | 命令或操作 | 日期 | 结果 | 证据
未运行项必须写未运行及原因, 禁止填通过。
## 结果统计
声明范围内: 风险样本数、正常样本数、误报数、漏报数。
范围外: 不支持数、信息不足数、解析异常数、超时数。
## 回归
MySQL/TiDB 固定语料的差异清单与人工确认结论。
## 限制
未覆盖语句、未验证版本、模式差异、样本规模、环境差异。
## 缺陷
编号、最小反例、影响、责任人、修复提交、复测证据。
## 结论
仅对已记录目标和能力给出结论; 支持声明由责任人确认。
```

7.5 步骤五：让 Review AI 找反例，再由人负责结论

代码审查应围绕真实差异和业务后果展开。让 Review AI 读取需求、设计、规则规范、测试报告及当前差异，并优先寻找会使错误 SQL 被放行的路径，例如解析异常被 catch 后返回空结果、未知目标被默认路由到 MySQL、规则异常被忽略、前端只依据发现项数量展示成功。每条发现都应写明位置、触发输入、实际行为、期望行为和影响，没有具体反例的猜测应标为待验证。AI 可以帮助扩大检查视角，但业务责任人与代码维护者仍要核实发现。

审查报告还要说明没有覆盖什么，例如审查者没有目标实例权限，不能验证数据库行为。不要用“未发现问题”替代“所有问题不存在”，也不要因为测试通过就跳过逻辑检查。修复后应由开发者说明差异，再运行直接相关的检查；已有必要检查通过后，无需反复扩大到无关测试。审查闭环完成时将高影响问题的修复提交与复测证据关联，低影响建议可进入技术债。若阻断问题未关闭，发布结论应明确保持阻断，而不是把问题藏在长篇附注里。

GAUSS-001 代码审查报告
审查范围: 列出提交、文件和相关文档
审查目标: 失败关闭、目标路由、规则语义、旧目标兼容性、敏感信息处理
发现模板
编号: REV-001
位置: 真实文件及行号
触发条件: 给出可复现输入与目标
实际行为: 依据代码或运行证据描述
期望行为: 引用需求/设计/规则规范
影响: 误放行、误报、兼容变化或可维护性问题
证据: 测试、日志摘要或最小反例
状态: 待核实/已确认/已修复/复测通过
典型反例
catch 解析异常后返回 findings=[], 调用方将其当作通过。
正确处理: 保留 UNSUPPORTED 或明确错误状态, allowed=false。
限制与结论
列出未审查能力; 未关闭阻断项使发布保持阻断。

第八章 发布验收与完整交接

发布不是把代码复制到生产环境，而是把有边界的能力交付给能够承担责任的人。下面以八个阶段展示完整闭环，每阶段都给出输入、产物和证据。实际团队可合并会议或缩短周期，但不能删除必要的判断。所有教学表格中的“待填写”“未运行”都应在真实项目中依据事实更新；本手册没有实际接入数据库、运行生产审核或完成灰度发布，不能作为 GAUSS-001 已成功上线的证明。

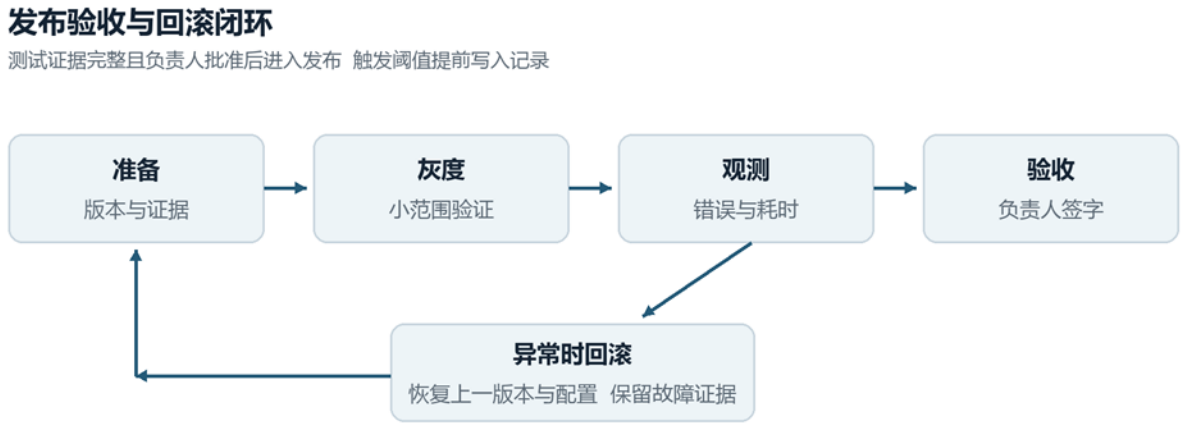


图 8-1 发布阶段的灰度 观测 验收与回滚流程

8.1 阶段一：需求确认与责任分配

输入是用户问题、现有平台约束和目标实例登记。产品负责人应把业务结果转成可观察条件，并请数据库负责人确认产品、部署、版本和模式。输出是经过评审的 requirement.md、范围清单和责任分配，而不只是会议纪要。对于还未确认的目标字段，应明确影响哪些后续工作：接口骨架可以继续，支持声明和目标集成不能提前结束。人工签字可以采用团队现有的评审系统或审批记录，关键是有可追溯的确认者、版本与日期。

阶段证据包括需求评审记录、脱敏目标信息、每条验收条件对应的样本计划。执行时逐条问“如果这项失败，使用者能否看见，团队能否判断”，将模糊词替换成状态、输入和预期输出。责任分配要具体到人或明确岗位，不能只写“产品 AI 负责”，因为 AI 不承担企业发布责任。退出标准是范围与目标足够明确，待决问题有处理路径；若业务方临时扩展到更多版本，应回到本阶段调整范围和计划，而不是让开发默默承担。

8.2 阶段二：方案评审与风险边界冻结

输入是已确认需求、仓库结构和能力矩阵。架构负责人组织评审接口、状态模型、解析器来源、规则边界和旧目标影响，输出 design.md、ADR 与修改清单。评审时沿着流程图检查每个失败出口，特别关注解析器缺失、规则无法求值和异常处理。默认离线审核要落实为代码与权限边界，不能只在文档写一句“只读”。需要数据库元数据时，另列查询许可、账户权限、超时与资源上限，并标出它不是本次最小离线能力的必要条件。

阶段证据包括方案版本、评审意见、未解决风险和对应负责人。冻结并不代表方案永远不能改变，而是避免并行实施时每个人使用不同契约。公共接口变化要记录影响范围，让开发和测试基于同一个版本工作。退出标准是关键风险有决定或明确阻塞，未知能力有诚实的结果表达。若后来发现解

析器不满足目标模式，应更新 ADR 与能力矩阵，缩小或暂停能力声明；不能用“架构已经通过”作为继续执行错误方案的理由。

8.3 阶段三：实现与持续记录

输入是设计基线、任务计划和语料规范。开发者或开发 AI 按小增量实现接口、适配器、解析器和规则，每次提交对应明确需求与测试。输出是可审查差异、更新后的 status.md 与能力矩阵。实现时先保持失败关闭，再逐项开放有证据的能力，避免为了演示效果先返回通过。旧数据库适配器应依据实际差异决定是否修改；无关重构、依赖大升级和格式清理宜拆成独立任务，使评审者能够看清本次行为变化。

阶段证据包括代码提交、开发检查摘要、规则规范和未验证项。每完成一项，应说明它解决哪条验收条件，哪些条件仍依赖真实环境。若开发途中遇到目标信息缺失，可以继续独立的输入校验与契约工作，同时把阻塞写入状态文件。退出标准是约定范围的实现可运行、关键边界有直接检查、任务状态与代码一致。不能因为所有计划文件都存在就报告开发完成，也不能把 AI 生成的日志描述当作真正执行过的命令输出。

8.4 阶段四：测试与支持范围核验

输入是候选提交、固定语料、目标环境和验收条件。测试负责人运行必要的单元、契约、MySQL/TiDB 回归及目标集成，输出 test-report.md 与缺陷清单。测试报告应区分通过、失败、跳过和未运行，列出实际目标与环境。没有真实 GaussDB 环境时，可以完成局部检查，但目标支持结论仍应保持待验证。对于性能要求，记录业务负载、样本选择、测量范围和失败请求，避免只展示单次快速结果。

阶段证据是可以复现的命令或操作、结果摘要、报告路径、提交标识和目标登记。测试负责人核对能力矩阵的每一项是否有对应证据，不能拿一个总的测试通过数字替代覆盖判断。发现误报和漏报时，应单独记录其影响和最小反例，并为修复添加回归样本。退出标准是阻断缺陷已关闭，声明范围内的条件有证据，剩余限制对产品负责人可见。样本没有覆盖的语法要保持未支持，不在发布文案中扩展为全量兼容。

8.5 阶段五：独立审查与发布决策

输入是候选差异、需求、设计和测试报告。审查者先核对实际代码与文档是否一致，再尝试构造能绕过规则的反例，输出 review.md 和问题闭环记录。重点查看状态丢失、空结果误放行、默认目标回退、异常吞掉和敏感信息泄漏。审查者应尽量独立于实现者，AI 角色切换只是辅助方式，不能代替真实的职责区分。若审查依赖同一份未经验证的假设，即使多个 AI 都赞同，也不会产生额外事实证据。

阶段证据包括具体发现、修复提交、复测结果与发布条件。发布负责人依据业务价值、覆盖范围和剩余风险作决定，明确哪些目标可以开放，哪些功能继续关闭。AI 可以整理风险和建议，但不替代企业授权的发布批准。退出标准是阻断问题已处理，支持范围表述与证据一致，灰度与回滚方案可以执行。若仍存在高影响漏报或目标集成未完成，应保留阻断状态；延期理由应写清影响和补证路径，避免一句“待优化”让风险失去责任人。

8.6 阶段六：灰度、观测与回滚演练

输入是批准的候选版本、灰度范围、监控指标和回滚方案。先在隔离环境演练配置开关、旧版本恢复和结果兼容，再按小范围用户或目标配置开启。输出是灰度记录、监控结果和异常处置记录。适配器开关应能快速关闭新能力，关闭后明确返回不支持或恢复既有流程，不能悄悄把 GaussDB 请求转给其他数据库引擎。灰度期间至少观察请求量、审核完成比例、不支持比例、错误、超时与用户反馈，并根据预先约定的基线判断异常。

阶段证据包括启用时间、范围、负责人、指标窗口和开关状态。回滚要说明触发条件、操作步骤、验证方式和数据影响；离线审核虽然不执行 SQL，仍可能存在配置与结果存储变化，应检查旧版本能否读取新记录。发现异常时先停止扩大范围，再依据影响关闭功能或恢复版本，保留最小必要证据用于分析。退出标准是观察窗口满足团队约定，关键指标没有未解释异常，回滚路径已经演练。没有执行灰度时，报告只能写“计划”，不能写“灰度正常”。

8.7 阶段七：业务验收与知识沉淀

输入是测试、审查和灰度证据，以及原始验收条件。产品负责人逐条验收，而不是只看新页面是否出现了 GaussDB 选项。验收应包括不支持语法的显示、风险发现的解释、旧目标的行为，以及目标版本条件是否清楚。输出是验收记录、实际开放范围和剩余问题。用户在演示中看到一条 SQL 返回结果，只能证明该场景被展示，不能替代批量语料与真实环境报告。验收记录需要引用同一个提交和目标登记，避免验收后代码变化却沿用旧结论。

阶段证据完成后，将稳定结论写入 .ai/knowledge、目标差异写入 .ai/database、最小 SQL 反例写入 .ai/sql-case，并通过索引相互引用。知识库应记录来源日期、适用版本、验证环境和何时复查，不把一次测试结果写成永久规律。发现的限制可以形成后续任务，但不能从交付报告中消失。退出标准是业务条件逐项有结论，文档反映实际能力，后续改进有负责人。教学案例到这里仍只是流程说明；读者必须根据自身执行证据填写验收结果。

8.8 阶段八：完整交接与下一轮任务启动

输入是最终代码、所有报告、配置和未完成事项。交接负责人创建 .ai/tasks/GAUSS-001/handoff.md，让没有参与聊天的维护者能够接手。交接应说明部署方式、目标范围、开关位置、排障入口、回滚步骤、已知限制和证据路径。保留必要结论与索引，避免复制大量无关聊天。接手者先核对版本，再用一个正常、一个风险、一个不支持的样本进行约定的冒烟检查；若真实能力尚未开放，应核对 UNSUPPORTED 行为与界面解释。

阶段证据包括交接检查记录、维护责任人和下一轮任务清单。下一轮扩展例如增加语句类型或支持另一模式，应重新建立目标、语料和验收条件，不直接复制“已支持”标签。交接完成的标准是维护者能找到证据、解释状态、定位问题并执行授权范围内的恢复操作。若任务仍有前置缺口，交接应明确标为未完成或待验证，不能为了结束项目把状态改成成功。最终报告用具体事实描述成果，例如“完成接口骨架与失败关闭测试”，比泛称“GaussDB 支持完成”更可靠。

```
# GAUSS-001 完整交接
## 1. 本次成果
填写实际行为；教学骨架尚未提供真实方言审核能力。
## 2. 版本与目标
代码提交：待填
产品/部署/内核版本/模式/参数：引用已确认登记
```

3. 开放范围
逐项列出已验证语句与规则；其他能力保持不支持。
4. 部署与配置
部署入口、配置开关、默认值、变更负责人；不包含密钥。
5. 证据索引
requirement.md、design.md、plan.md、status.md、test-report.md、review.md
能力矩阵、ADR、语料版本、灰度记录、验收记录。
6. 排障
先核对目标 → 查看审核状态 → 查脱敏任务日志 → 定位样本与规则。
输入无效、不支持、解析异常、规则异常、超时分别处理。
7. 回滚
触发条件：待团队确认
操作：关闭适配器开关或恢复批准的旧版本
验证：状态契约、旧目标回归、记录读取与监控恢复
8. 限制和未完成项
事项、影响、负责人、下一步、预计复查点；未运行写未运行。
9. 接手确认
维护负责人：待填；核对日期：待填；冒烟结果及证据：待填。
10. 下一轮
新增版本、模式或规则时重新建立目标登记、语料和验收条件。

表 8-1 八阶段交付的最小证据链

阶段	主要产物	禁止的跳步
需求	requirement.md、目标登记	目标未知却承诺支持
方案	design.md、ADR、能力矩阵	未经证据复用其他方言
实现	代码差异、status.md	空结果代表通过
测试	test-report.md、语料	未运行写成通过
审查	review.md、问题闭环	仅凭 AI 赞同放行
灰度	开关、监控、回滚记录	没有演练却声称可回滚
验收	条件逐项结论、知识库	展示成功代替完整验收
交接	handoff.md、责任人	未完成任务标记成功

第九章 Codex 与 VS Code 日常操作

9.1 打开正确的项目根目录，并建立第一次检查点

操作从选择文件夹开始。把教学项目放在一个名称清楚的目录，例如 sql-audit-platform，再在 VS Code 中打开包含 pom.xml、README.md、AGENTS.md 和 .ai 的这一层。不要只打开 src/main/java，也不要把装着多个无关项目的上层目录当作当前项目。打开之后先对照左侧文件树，确认主代码位于 src/main/java/com/example/audit，测试位于 src/test/java/com/example/audit，SQL 样本位于 src/test/resources/sql。这个检查解决的是工作对象问题：同名文件可能存在于多个副本，AI 即使正确执行指令，也可能把修改写到错误副本，因此根目录必须在人与 AI 开始合作前就固定下来。

接着在项目终端核对当前分支、工作区差异和最近一次提交。已有未提交修改时，先判断它们属于谁、是否与本次任务相关，并把结论写入任务状态；不能为了让状态好看而删除别人的工作。第一次练习最好从可构建的提交开始，记录提交标识与 Maven 验证结果，随后再进入需求分析。检查点的意义是给比较和恢复提供参照：任务结束后可以说明增加了哪些行为，也能区分旧问题与新问题。IDE 官方说明支持在编辑器中引用打开文件、选择代码并查看修改；本书把这些能力用于聚焦具体文件，不假设所有项目内容都会自动成为有效上下文。[S2]

```
# 在项目根目录运行；以下命令只检查当前状态
git rev-parse --show-toplevel
git status --short
git branch --show-current
git log -1 --oneline
mvn -B -ntp verify
```



图 9-1 首轮操作示意：核对根目录 → 核验规则 → 勘察代码 → 建立任务检查点 → 查看差异

9.2 核验 AGENTS.md 的作用范围，而不是凭感觉判断生效

在根目录 AGENTS.md 中保留执行约定和入口索引，把长篇背景留在 .ai 文件中。Codex 的约定文件发现机制会按范围组合说明，存在 AGENTS.override.md 时还可能覆盖同层普通文件；具体发现顺序与大小限制应按所用版本官方文档核对。[S1] 对新手更可靠的操作是，在新会话第一条消息中要求 AI 列出当前采用的说明文件、复述三条关键限制，并说明还要读取哪些项目资料。仅让 AI 回答“我已了解规则”没有诊断价值；要求它给出文件路径、规则内容以及适用目录，才能发现打开了错误根目录、文件为空、临时覆盖文件未撤销等实际问题。

这里必须分清工具约定与团队自建目录。.ai/AI_CONTEXT.md、.ai/AI_RULES.md、.ai/AI_WORKFLOW.md 是本书推荐的团队文件，不因目录叫 .ai 就获得自动加载能力。根目录 AGENTS.md 应明确写出读取入口，例如“开始实现前读取任务的 requirement.md、design.md、plan.md”。当目录较大时，还应要求 AI 只读取与当前任务有关的知识条目，并解释选择依据。规则调整后重新开启合适的会话并再次核验；涉及平台具体重载行为时查当前说明。核验结果写进 status.md 的环境栏，后续接手者便能知道当时使用的规则，而不需要猜测会话里隐藏了哪些条件。

```
首轮规则核验 Prompt：
请先报告当前项目根目录、当前分支以及未提交修改摘要。
列出你实际读取或当前可用的说明文件路径；不要把未读文件说成已读。
复述 AGENTS.md 中与本次任务有关的范围限制、验证命令和交付要求。
读取 .ai/AI_CONTEXT.md 与 .ai/tasks/GAUSS-001/status.md，列出事实、假设和缺失信息。
此轮先完成勘察，不修改代码。
```

9.3 用首轮勘察 Prompt 建立代码地图与任务范围

首轮勘察要让 AI 证明它理解了真实调用链。把“了解一下项目”改为可以核验的要求：定位 HTTP 入口、审核服务、数据库类型选择、解析器工厂、规则执行器、结果对象与对应测试，并为每个判断附文件路径和关键符号。要求它从一个已存在的数据库类型开始追踪，例如在教学代码确有 MySQL 支持时沿实际入口追到规则输出；没有该支持时应报告缺口，不能补成项目事实。输出一张简明代码地图后，由开发者抽查两三条调用关系。这样的勘察比一次性上传全部代码更有用，因为它把“项目大概如此”转化为可以在编辑器中跳转验证的证据。

地图确认后，再读取 GAUSS-001 的需求、设计和计划。要求 AI 输出拟修改文件清单、每个文件的改动理由、依赖顺序与验证方法。若它提出修改公开接口、升级解析器或增加运行依赖，应在设计里解释触发原因和兼容性影响，不能把这些动作夹在普通实现中。开发时一次推进一个可验证切片，例如先增加数据库类型路由的测试，再增加适配器，再连接规则。每个切片结束后在编辑器查看差异，并检查是否出现无关格式化或旧逻辑变化。通过这种节奏，范围管理成为每天的普通操作，而不是等到 PR 累积几十个文件后才开始追问。

```
项目勘察 Prompt：
从现有接口追踪一次 SQL 审核请求，列出入口→服务→类型分派→解析→规则→结果的实际文件与符号。
定位现有测试如何装载 src/test/resources/sql 中的样本。
只把代码或测试能够证明的行为写为事实；未知行为单独列出。
读取 .ai/tasks/GAUSS-001/requirement.md、design.md、plan.md。
输出最小改动范围、三个主要风险及每个风险的验证办法。
不要在勘察阶段进行实现或升级依赖。
```

9.4 把会话进展写成可恢复的任务状态

长会话不能代替任务记录。准备结束工作、切换角色或换电脑之前，更新 .ai/tasks/GAUSS-001/status.md 和 handoff.md。status.md 描述当前阶段、已完成内容、未完成内容、阻塞条件、最后验证结果和下一步；handoff.md 提供接手者需要的路线，包括当前分支、提交标识、关键文件、已经否决的方案及否决原因。状态应写成具体行为，例如“数据库类型路由测试通过，解析器兼容样本尚未运行”，不要写“进展良好，基本完成”。可恢复的标准是另一名工程师或新会话仅凭仓库记录，就能判断下一条操作，并且不会重复已经完成的工作。

恢复时先比较记录与工作区。记录说测试通过，不代表当前文件也经过同一轮验证；如果最后验证之后又有代码变化，需要对受影响部分补验。记录说分支是某个名称，也不代表当前电脑已切到该分支。因此新的 AI 会话要先检查提交与差异，再读取任务文件，再用自己的话说明“目前在哪一步、下一步做什么、依据是什么”。开发者确认这个复述符合实际后继续实现。若会话被中断，不必重述整段聊天，只提供任务路径和恢复目标即可。把关键背景固化为小文件，会降低上下文成本，也让团队的工作能够脱离某个账号或某台电脑继续推进。

```
# GAUSS-001 交接记录
更新时间: YYYY-MM-DD HH:mm, 团队统一时区
当前阶段: 实现中 / 验证中 / 等待评审
分支与提交: 填写实际值
工作区差异: 列出未提交文件及归属
已完成: 逐条写可核验行为
最后验证: 命令、时间、提交、结果、报告路径
未完成: 解析样本、旧数据库回归等
下一步: 明确第一条操作与预期结果
已否决方案: 方案、理由、证据
恢复前必读: requirement.md、design.md、status.md
```

9.5 分支、worktree 与多设备同步的日常纪律

分支用于表示一条变更历史，worktree 用于让同一仓库拥有多个独立工作目录。对单人初学者，一个任务分支通常已经足够；当一个人同时处理缺陷和功能，或者多个 AI 实现容易改到相同文件时，独立工作目录更便于隔离。不要把两个会话同时指向同一目录后期待它们自动协调。开始任务前记录该工作目录的实际路径与基础提交；合并前比较基础提交是否已经落后。Codex 对受管理 worktree 的创建、交接和恢复有产品自己的行为，命令行创建的 Git worktree 又有不同的管理边界，因此先确认当前采用哪一种方式，再执行清理或转移。[S3]

多设备同步以仓库提交为依据，而不是把聊天窗口当作同步工具。电脑 A 结束工作时，检查差异，提交与任务相关的代码和 .ai 记录，再推送任务分支；电脑 B 获取该分支后核对提交，恢复依赖与本地配置，运行规定验证，再继续任务。密钥、生产连接串和私有数据库导出不应跟随教学文件提交；在项目中保留不含敏感值的配置示例，让接手者知道需要哪些变量。若两台电脑同时工作，使用不同分支并通过正常评审合并。出现冲突时逐段理解两方意图，尤其检查需求文档与实现是否配套，不能仅以保留“最新一边”作为解决标准。

表 9-1 不同工作方式的选择与恢复要求

方式	适用情况	开始前检查	结束时保留
单目录任务分支	一个任务、少量修改	根目录、分支、旧差异	提交、状态、测试报告
独立 worktree	并行任务或易冲突改动	实际路径、基础提交、配置来源	分支或可恢复提交、交接记录
多设备接续	同一任务换电脑	拉取后的提交、本地环境	已推送提交、配置示例
多角色评审	实现与审查分开	同一变更基线、可访问证据	评审发现、修复与复验结果

9.6 工具中立迁移：把协议带到 OpenCode 或其他助手

迁移工具时优先保留团队的输入输出协议。需求、设计、测试报告、ADR 与 SQL 样本都属于项目资产，可以继续放在同一仓库中；模型名称、登录方式、允许执行的操作和说明文件发现机制属于具体工具，需要按目标工具当前文档重新确认。不要因为某个工具名称含有“Code”，就假定它与 Codex 自动读取相同文件，也不要照搬未经核验的按钮名称、配置键或快捷键。第一次迁移先用只读任务，让新工具报告当前目录、读取指定文件、追踪一条调用链并给出证据；其结果通过人工抽查后，再进入低风险代码修改。

建立 .ai/prompts/bootstrap.md 作为通用启动模板，内容只表达任务协议：读取哪些文件、区分事实与假设、先报告范围、修改后如何验证、交付哪些记录。工具如果支持仓库说明文件，就按照它自己的文档把这份协议接入；不支持时在每次会话中明确粘贴或引用必要部分。迁移验收以同一组教学任务为准，例如恢复 GAUSS-001 状态、识别一个不受支持的语法、说明旧数据库回归范围，而不以回答篇幅或主观流畅度打分。这样可以比较真实工作质量，也能在工具发生变化时继续维护研发体系。

第十章 项目知识的长期维护

10.1 建立知识索引与统一记录头

长期维护先解决“在哪里找到可信资料”。在 .ai/knowledge/INDEX.md 中列出技术债、事故复盘、常见实现模式和验证方法；数据库知识放在 .ai/database，SQL 案例放在 .ai/sql-case，架构决策放在 .ai/architecture/adr。索引只保留标题、主题、状态、维护人、适用版本和相对路径，避免把每篇正文重复复制进去。任务完成时，文档角色检查有没有形成可复用经验，再决定新增条目还是更新旧条目。一次普通修复未必值得新建十个文件；只有会影响后续判断、可以复现或能够避免再次返工的知识，才进入长期索引。

每个条目使用统一记录头，至少包括创建日期、最后核验日期、维护人、适用范围、证据来源和状态。状态可以使用“待核验、已验证、部分适用、已过期、已替代”，并要求 AI 在引用时带上状态。这样处理的原因是软件版本、数据库产品形态与部署限制会变化，一篇曾经正确的文章可能在新版本里失效。知识库并非只增不减的聊天收藏夹，而是可审查的项目资料。检索到旧条目时，先检查是否覆盖当前版本与场景，再引用具体结论；没有证据支持的经验可作为待验证假设，不能自动升级为当前项目规则。

```
# 知识条目标题
ID: KB-YYYY-NNN
状态: 待核验 / 已验证 / 部分适用 / 已过期 / 已替代
维护人: 岗位或实际人员
适用范围: 模块、数据库产品形态、版本、部署条件
创建日期: YYYY-MM-DD
最后核验日期: YYYY-MM-DD
证据: 官方资料 URL、测试路径、提交或报告
结论: 可以直接使用的行为说明
限制: 未覆盖情况与禁止外推的边界
复现: 步骤、输入、预期、实际结果
关联: 任务、ADR、替代条目
```

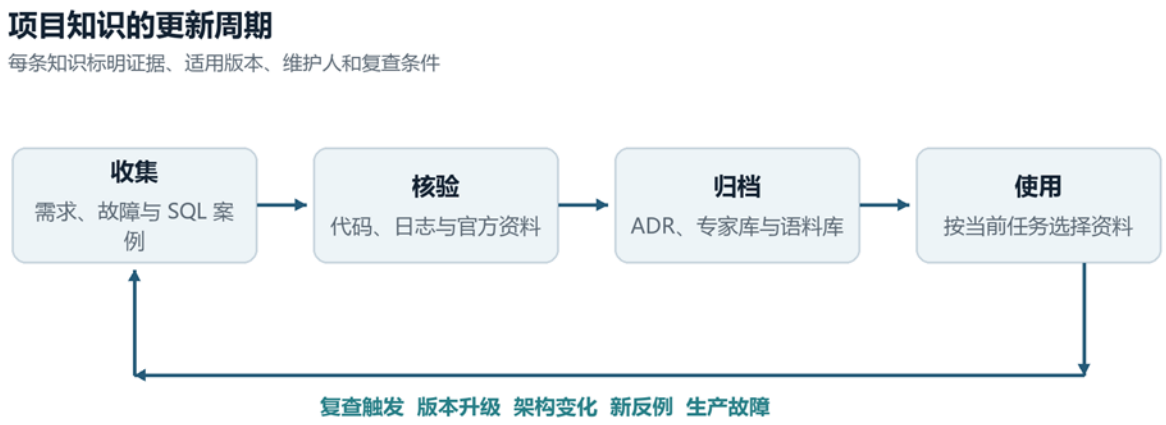


图 10-1 知识生命周期：任务证据 → 人工核验 → 分类入库 → 使用前检查 → 定期复验 → 替代或归档

10.2 技术债不是愿望清单：记录影响、触发条件与偿还证据

技术债条目可以存入 .ai/knowledge/technical-debt/DEBT-001.md。不要只写“代码需要重构”或“解析器不好用”，要描述现有设计在什么条件下导致什么成本。例如，类型分派使用重复分支，新增数据库时需要同时修改多个位置，漏改会出现配置与实现不一致；证据是本次任务涉及的实际文件与测试。然后记录影响范围、当前补救措施、建议方案、预计工作量和触发偿还条件。触发条件应可观察，例如第三个数据库适配器上线前、某类缺陷再次出现时，或者审核延迟超过已确定目标时，而不是“以后有空再做”。

排序时同时考虑业务影响、发生概率、变更难度和时机。对本次需求没有直接影响的债务，AI 可以给出建议并登记，不应借实现功能之名扩大改动；对会阻止正确实现的债务，则应在设计阶段明确纳入范围。完成偿还之后，不要只把状态改为关闭，还要附上变化前后的证据，如分派位置减少、回归样本通过、重复缺陷不再出现。企业可每两周安排一次短审查，只挑少量高影响条目进入下轮计划。这个节奏让技术债既不会被遗忘，也不会反过来吞掉需求交付的时间。

表 10-1 技术债记录的最小字段

字段	填写示例	为什么需要
现状与证据	类型分派重复，列实际路径	避免凭印象启动重构
业务影响	新增数据库漏接入的风险	帮助产品理解优先级
当前措施	新增路由契约测试	控制偿还前的风险
偿还触发	新增下一种数据库之前	把愿望转成计划条件
完成证据	提交与回归报告	确认问题实际得到改善

10.3 事故库：先记录时间线，再追问为何防线失效

事故库放在 .ai/knowledge/incidents，记录已经发生的异常以及有代表性的未遂故障。先还原发现时间、影响开始时间、涉及版本、用户表现、处置动作和恢复结果，再区分直接原因、促成条件与未能发现问题的防线。例如某类 SQL 被标记为安全，但实际语法无法解析，直接原因可能是异常路径返回了默认通过；促成条件可能是缺少失败样本；防线缺口则可能是测试只覆盖成功路径。复盘应引用日志片段、测试或提交，但先移除连接串、账号、客户信息与真实业务数据，保留足够分析所需的结构。

把行动项写成可以追踪的工作。每项注明维护人、期限、验证方法和关联任务，随后由任务记录回写完成证据。不要把“加强意识”“以后注意”作为主要改进，它们很难证明是否生效。对于上面的默认通过问题，改进可以是统一返回解析失败状态、加入失败语料、增加契约测试并检查接口展示。AI 可以辅助归并时间线、寻找遗漏情形和提出候选原因，但最终原因需要由实际证据支持。复盘完成后还要更新对应规则或知识条目，让下一次实现能够读取到经验；单独存一篇事故文章而没有任何流程变化，通常无法防止重复事故。

```
# INC-001 SQL 解析失败被误判通过
状态：教学示例，待按实际事故填写
影响：请求范围、持续时间、用户表现
时间线：发现→隔离→定位→恢复→验证
直接原因：附日志、代码或测试证据
促成条件：样本、配置、发布等条件
防线缺口：为何测试或评审未发现
处置：实际采取的措施与结果
行动项：任务 ID、维护人、期限、验证方法
资料处理：敏感内容已移除，原始证据访问位置
关联知识：规则、ADR、语料条目
```

10.4 SQL 语料与数据库专家库：每个判断都带适用边界

SQL 语料既要保留能正确通过的语句，也要保留应拒绝、应警告、无法解析和暂不支持的语句。执行用样本放在 src/test/resources/sql，并由测试加载；说明放在 .ai/sql-case 中，以案例编号关联样本路径。每个案例记录数据库产品与形态、版本、前置表结构、SQL、预期结果、规则编号、证据和边界。真实业务 SQL 进入语料前先脱敏，字段名和常量替换后还应检查是否保留了原来的语法或性能特征。只收藏“经典慢 SQL”会偏向容易解释的情况，漏掉解析失败、空输入、多语句、注释、大小写及特殊标识符等真正容易破坏审核流程的边界。

数据库专家库按产品分别维护，但同一个产品名称也可能包含不同产品形态、兼容模式和版本。关于 GaussDB 的知识不能由 PostgreSQL 或其他产品经验直接代替。条目中应明确哪些结论来自该版本官方文档，哪些来自当前环境实验，哪些只是计划验证的推测；涉及连接驱动、方言特性或执行计划时还要注明环境条件。AI 回答数据库问题时先读取对应条目，再检查来源日期与版本，并报告未覆盖的边界。如果无法确认真实数据库环境，本书示例只演示审核适配的工程方法，不宣称已经支持全部 GaussDB 语法，更不把静态审核结果解释为数据库实际执行必然安全。

表 10-2 SQL 案例与数据库知识的证据分层

层级	记录方式	可以用于	不能推出
官方资料	产品、版本、URL、章节、核验日期	核对正式声明的行为	本项目实现已完成支持
环境实验	版本、配置、输入、输出、步骤	证明该条件下的实际行为	其他部署必然一致
仓库测试	样本路径、断言、提交、报告	证明当前代码的回归结果	真实数据库全语法兼容
待验证经验	假设、来源、验证计划	辅助设计验证路径	作为上线依据

10.5 ADR 模板与生命周期：保留做决定时的条件

ADR 是架构决策记录，放在 .ai/architecture/adr，并使用连续编号，例如 0001-database-adapter-boundary.md。一份 ADR 只回答一个足够重要的问题，例如数据库方言差异放在适配器层还是散落到通用规则中。正文先写上下文与约束，再列候选方案、选择理由、影响和验证方法。理由要包含被放弃方案的真实优缺点，不能写成“采用方案 A，因为方案 A 更好”。重要选择往往是权衡：隔离适配器提高可维护性，但增加接口设计和契约测试成本。把这些条件记录下来，后来的人才知道何时可以沿用，也知道出现什么变化后需要重审。

生命周期从提议开始，经过审查后接受；实施并验证后更新结果，未来可被新 ADR 替代或明确废弃。已接受的记录不应被悄悄改写成仿佛一开始就知道最新结论，必要的小修订注明日期，实质变化创建新记录并互相链接。任务 design.md 引用相关 ADR，PR 检查是否遵守当前决策，知识索引标出当前有效记录。AI 可以拟写候选 ADR，也可以发现实现与决策不一致，但不能仅凭生成了一份文档就声称团队已接受方案。接受人、时间和依据要留在记录中，让架构治理依赖透明证据，而不依赖某段难以寻找的聊天。

```
# ADR-0001 数据库差异的适配器边界
状态: 提议 / 已接受 / 已替代 / 废弃
决策日期: YYYY-MM-DD
接受人: 实际负责角色
上下文: 为什么现在必须决定
约束: 兼容性、时间、运行环境、团队能力
候选: A、B，各自优点、成本与风险
决定: 选择与关键理由
影响: 哪些文件、接口、测试、运维行为变化
验证: 验收证据及结果
重审条件: 版本、规模或需求变化
关联: GAUSS-001、替代 ADR、代码提交
```

10.6 防过期、去重复与隐私维护的固定动作

每月由维护人检查知识索引，优先复验依赖外部版本、曾经作为上线依据或者已超过核验期限的条目。发现过期时先标记并写明原因，然后安排复验、替代或归档；不要删除后失去历史线索。发现多篇资料重复描述同一事实时保留一个主条目，其他文件通过链接引用，避免下一次修改只更新其中一份。新增知识时检查索引是否已有相关主题，检索时限制到与任务有关的目录与状态。把使用次数高但频繁导致误解的条目加入维护清单，知识质量便可以通过真实使用反馈改善，而不只是通过统计文件数量评价。

隐私维护贯穿入库与使用。仓库知识只保存任务必要的事实，事故原始日志、完整数据库转储、个人信息和凭证应按企业已有规则存放在有权限控制的位置，知识条目记录可访问的证据位置与获取责任人即可。培训样本用合成数据并标注教学用途。AI 准备把资料送入外部工具时，需要符合组织确定的资料范围与授权，不可因为文件位于 .ai 就推定允许外发。维护人抽查新增条目有没有真实证据、有没有泄露无关信息、有没有把假设写成结论。三个检查形成小而稳定的门槛，比一开始建立复杂却无人执行的知识平台更容易落地。

第十一章 CI 自动化与团队控制

11.1 先自动化可重复验证，再引入 AI 判断

自动化的第一项工作是让不同开发者能够运行相同验证，而不是让 AI 每天自由生成代码。对本书 Java、Spring Boot、Maven 教学项目，先确认 pom.xml 固定必要依赖与插件版本，README.md 写清 JDK、Maven 和本地配置要求，再把约定的验证入口收敛为一条可重复命令。构建、单元测试、契约测试、样本回归与静态检查应由真实工具给出退出状态和报告。若项目尚未配置某项检查，就记录未覆盖范围，不要因为命令名称叫 verify 就假定所有测试都已执行。确定性底座的作用是让每次变更有统一比较依据，也让 AI 的解释能够被真实结果约束。

本地执行之后，保存命令、时间、提交标识、通过与失败项目以及报告路径到 test-report.md。AI 可以根据日志定位失败、提议补充案例或总结验证范围，但只有实际执行得到的结果才能写为通过。测试被跳过、依赖下载失败、数据库环境不可用和测试断言失败是不同状态，分别记录后采取相应动作。建立验证入口时先在人工已知正确的提交上跑通，再故意破坏一个断言，确认流程确实会失败；随后撤销练习改动。这个小演练验证的是防线真的有效，避免团队把无测试、永远返回成功或错误忽略失败的脚本当成可靠质量检查。

```
# 示例 scripts/verify.ps1：本地统一验证入口
$ErrorActionPreference = 'Stop'
if (-not (Test-Path -LiteralPath 'pom.xml')) {
    throw '请在包含 pom.xml 的项目根目录运行。'
}
mvn -B -ntp verify
if ($LASTEXITCODE -ne 0) {
    throw "Maven 验证失败，退出码：$LASTEXITCODE"
}
# 是否包含集成测试，取决于 pom.xml 中实际配置的插件和生命周期。
```

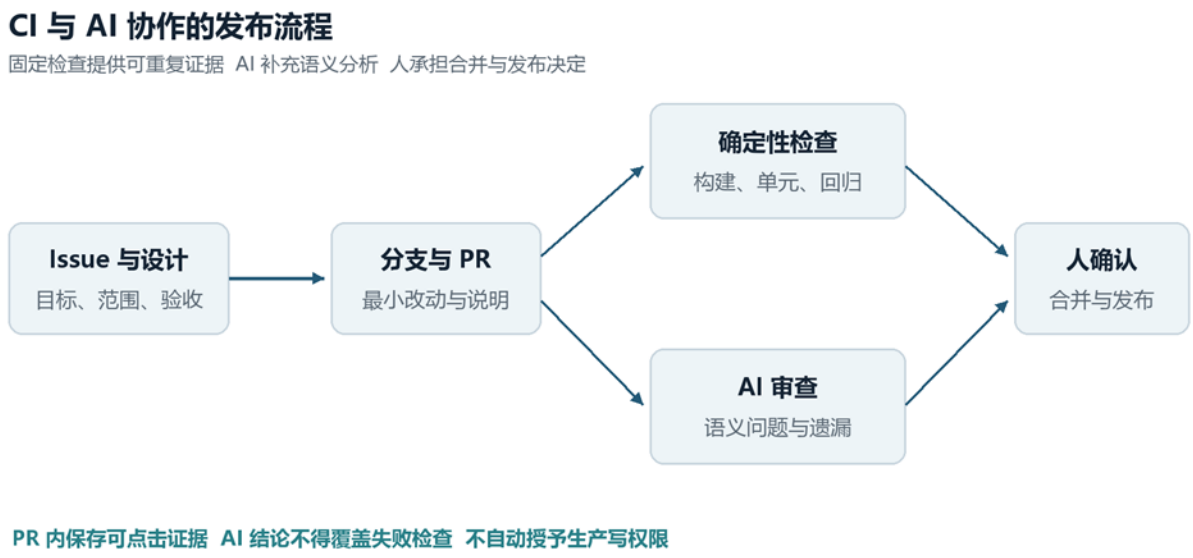


图 11-1 团队交付流程：Issue → 设计 → 任务分支 → 本地验证 → PR → CI 与评审 → 人确认 → 发布与知识回写

11.2 GitHub Actions 最小可复制工作流与权限

下面工作流存为 .github/workflows/verify.yml，面向已经能在本地运行 Maven 验证的教学仓库。它在 PR、main 分支推送及手动触发时检出代码、配置 JDK 并执行 Maven verify，权限仅授予读取仓库内容。示例不依赖 AI 服务或数据库密钥，便于团队先验证持续集成链路。[S_GH1] JDK 17 是教学假定，必须与项目 pom.xml 的实际编译目标一致；main 也应改成实际主分支。工作流通过代表配置中的检查通过，不能代替业务验收、真实数据库兼容验证或发布确认。首次添加后提交到试点分支，观察一次成功和一次有意失败的练习结果，再设置团队需要的合并检查。

```
name: Verify
on:
  pull_request: {}
  push:
    branches: [main]
  workflow_dispatch: {}
permissions:
  contents: read
jobs:
  verify:
    runs-on: ubuntu-latest
    timeout-minutes: 15
    steps:
      - name: Checkout
        uses: actions/checkout@v7
        with:
          persist-credentials: false
      - name: Set up Java
        uses: actions/setup-java@v6
        with:
          distribution: temurin
          java-version: '17'
      - name: Verify Maven project
        run: mvn -B -ntp verify
```

本书在核验时，官方动作仓库示例使用 checkout@v7 与 setup-java@v6；这只是当前可核对的示例版本，企业启用时还要检查运行器要求、审查变更，并将动作固定到批准的完整提交标识以便复现。[S_GH3][S_GH4] GITHUB_TOKEN 的权限应按任务所需最小化；普通验证不需要仓库写权限。[S_GH2] 若以后加入创建 Issue、评论或发布动作，应单独说明权限变化、输入来源和人工责任。对来自不可信分支的代码，不把拥有生产凭证的作业直接用来执行它。数据库集成测试应采用专用测试环境和必要权限，与生产发布分开配置，让失败和授权边界都清楚可见。

11.3 Issue 到发布：每一次交接都带可核验材料

Issue 起点至少包含问题、业务目标、范围、验收案例和未决事项，随后产品角色形成 requirement.md，架构角色形成 design.md，开发负责人形成 plan.md。这里的角色可以由同一个人协调多个 AI 会话，不要求企业一开始购买六套工具。实现会话只处理已确定范围，完成后创建包含行为变化、验证结果与风险的 PR。评审者同时检查代码差异、需求、关键测试及知识变更。AI 评审可以提示异常路径或遗漏样本，但每个发现应给出触发条件、影响和证据，由负责人判断是否需要修复，不能以生成了一份长报告作为完成评审的标准。

合并与发布是不同阶段。合并前，约定检查通过、必要评审完成、未决事项可接受；发布前，确认目标环境、配置、数据库变更、回滚办法与负责人。AI 可以生成发布检查材料和说明，但企业最

终责任必须明确落到有权限的人员与现有发布制度上。发布后观察约定指标和错误日志，如果功能不符合预期，按照事先准备的恢复方案处理，并把事实写入任务记录。最后更新知识索引与 ADR 状态，关闭 Issue。这个流程没有把人插在每一个小动作之间，而是把人放在业务范围、重大设计和发布责任等真正需要判断的位置。

表 11-1 交付门槛与必须留下的材料

节点	材料	确认责任
开始实现	需求、设计、计划、范围	开发负责人及需要参与的业务方
创建 PR	代码差异、测试报告、行为说明	实现负责人
合并	CI、评审发现、修复证据	仓库维护者
发布	配置核对、回滚、观察指标	有发布权限的负责人
关闭任务	发布结果、知识更新、未完成项去向	任务负责人

11.4 自动日报：从事实生成摘要，保持提交可追溯

日报自动化先确定读哪些事实：当天完成的提交、PR 状态、任务 status.md、验证报告和阻塞条件。把日报模板放在 .ai/prompts/daily-report.md，要求按已完成、进行中、阻塞、下一步组织，并在每条进展后附任务编号或提交依据。对于代码已完成但尚未验证的内容，写为“待验证”；对于讨论中的方案，写为“候选方案”。禁止 AI 用聊天中的乐观描述替代仓库状态。团队也不应让日报创造额外工作：任务记录已经维护完整时，摘要从现有资料生成即可，开发者只核对异常与事实遗漏。

试点阶段先把日报生成到本地供团队审阅，再依据实际需要接入已有通知渠道。计划自动发送消息时需要明确接收人、频率、敏感内容范围和发送授权；生成文稿与外部发送是两个动作。日报应保持短，突出完成的行为和需要处理的阻塞，未变化状态可以省略。每周随机抽查几条日报能否追回真实提交与报告，发现夸大进展时改进提示词或输入清单。不要只评价“生成用了几秒”，还要计算核对时间与错误纠正成本；自动化如果让管理者更难识别未完成工作，就需要调整而不是继续扩大使用范围。

```
日报生成 Prompt：
读取今日相关任务的 status.md、test-report.md 和可访问的提交/PR 状态。
只生成有证据的进展，每条附任务 ID 或提交依据。
使用四栏：已完成、进行中、阻塞、下一步。
代码完成但未验证必须标注待验证；未读资料不得写成事实。
省略未变化的普通状态；突出需要负责人处理的阻塞。
输出供本地审阅的摘要，外部发送按团队单独授权执行。
```

11.5 评估质量、成本与安全，避免只计算生成速度

试点评估用相同难度任务比较 AI 辅助前后的完整交付过程，记录需求澄清、实现、测试、评审修复和交接所用时间。质量指标至少包含首轮验收通过率、评审发现的问题、上线后缺陷与恢复任务所需时间；成本包括工具费用、调用成本、等待、人工核对与返工。样本量较小时只报告事实与趋势，不宣称已经证明某个固定提升比例。对 SQL 审核平台，还要单独检查错误放行、错误拒绝和不支持状态是否清楚，因为普通测试通过率无法充分表达这些行为对使用者的影响。

安全控制与日常研发连接在一起：仓库不提交凭证，样本不含客户敏感信息，工具访问范围符合项目授权，外部内容中的指令不自动成为工作要求。把依赖更新、生产访问、发布和数据操作纳入已有责任边界，避免把它们隐含在“让 AI 完成任务”里。评估时记录权限请求为什么发生、是否必要，以及是否存在更小范围的方案。成本异常时先定位是否输入了重复资料、任务范围失控或反复验证同一结果，再选择缩小上下文、改善模板或调整任务分工。预算应该服务可靠交付，不能为了少一次调用而省略关键验证。

11.6 逐步升级自动化，并保留失败时的人工路径

成熟自动化从只读摘要逐渐升级到生成文档、执行验证、提出变更和创建待审 PR，每一层都先建立输入、输出、权限与失败处理规范。某层运行稳定之后再进入下一层，不要求一次性实现全自主研发。给作业设置超时、重复执行识别和日志保留规则，避免失败后无休止重复或者成功后重复创建相同材料。重要作业要能够由人手动重跑，并能明确判断此次运行对应哪个提交、哪些输入以及哪些结果。人工路径不是备用口号，而应有具体入口和负责人，停用自动化后团队仍然能够完成必要交付。

每次自动化变化先在试点仓库或教学分支演练失败场景，例如依赖下载中断、测试失败、权限不足、输入缺少需求或存在冲突。预期行为是输出明确状态并停止依赖它的后续操作，保留已经得到的证据，交给相应负责人处理。AI 不应把环境失败解释成业务通过，也不应擅自放宽检查让流水线变绿。把这些演练结果写入 .ai/knowledge/automation-runbook.md，定期依据真实故障补充。团队最终得到的是可以运行、可以解释、可以暂停和恢复的系统，而不只是一次成功演示的视频。

第十二章 企业落地路线图与演练

12.1 试点范围、职责与量化验收标准

选择试点时优先选有明确维护人、可本地构建、能够使用合成数据且需求规模适中的项目。SQL 审核平台可以从一个新增数据库适配的教学切片开始，不把历史重构、真实数据库全兼容和自动生产发布同时纳入第一轮。指定一名试点负责人协调任务，一名熟悉代码的工程师确认设计与评审，一名熟悉业务的人员核对验收案例；人手有限时可以兼任，但不同确认动作仍应在记录中区分。AI 角色承担资料整理、方案候选、实现辅助、测试分析和文档拟写，最终事实与责任留给指定人员。

开始前选定可统计的验收标准，例如任务记录齐全率达到全部试点任务、每个 PR 附实际测试依据、新会话恢复同一任务不超过团队设定时间、已确定高风险 SQL 样本全部有明确状态。速度指标使用人工总投入和从需求到验收的时间，质量指标使用漏检、误检、评审修复与回归结果。目标应根据现有基线确定，本书表格给的是可调整的试点目标，不是对任何企业都适用的承诺。固定统计方法比目标数字本身更重要：如果前后两轮任务难度不同，或者漏算复核时间，结论就难以指导后续投入。

表 12-1 试点验收指标示例（由团队按基线调整）

指标	建议目标	证据来源
任务记录齐全率	试点任务 100%有需求、状态和验证结果	任务目录抽查
PR 验证依据	每个试点 PR 附真实执行结果	PR 与 test-report.md
任务恢复时间	新会话 15 分钟内说明下一步	恢复演练记录
高风险样本覆盖	已确定高风险样本 100%有断言	样本索引与测试报告
人工投入变化	与同类历史任务比较并报告差异	工时与复核记录
外发与凭证错误	试点不出现未授权外发或提交凭证	审查与事件记录



图 12-1 九十天推进路径：前 30 天建立基线 → 60 天形成闭环 → 90 天验证可复制与治理

12.2 第一至三十天：把基础资料与验证跑通

第一阶段的重点是让少量任务能可靠完成。第一周整理项目根目录、README.md、AGENTS.md 与 .ai 基础文件，确认构建环境，创建第一条需求并完成一次只读勘察。第二周用一个小切片检验需求、设计、计划、实现、测试与评审的衔接，同时进行新会话恢复练习。第三周处理发现的问题，删去无用模板字段，补充真实缺少的规则和样本。第四周统计任务耗时、复核成本、遗漏与返工，形成第一份试点复盘。这个阶段不追求大量自动化；先证明文件确实有人维护、验证确实能够发现错误、角色交接确实能够继续工作。

三十天验收时抽取至少几个完整任务，逐项追查行为与证据是否配套。代码实现了某条规则，需求里应有对应目标，测试里应有关键案例，报告里应写明实际运行结果；计划里没有完成的事项必须有明确去向。若模板很完整但恢复仍然困难，说明字段缺少行动信息或者内容没有及时更新。若 AI 输出很快但人工花费大量时间查错，应缩小范围并改进勘察与测试环节。第一阶段只有通过这种抽查，才算从“会聊天写代码”进入“可以复用的研发流程”。未通过的项目继续修复基础，不急着推广。

12.3 第三十一至六十天：形成协作闭环与知识复用

第二阶段扩大到不同类型任务，例如新增规则、修复解析异常和调整接口表现，让流程承受不同输入。引入独立评审角色，要求评审者依据变更基线和需求检查行为，并把发现写成可复现条件。将本地验证接入 CI，安排一次失败演练，确认检查不会被绕过。同步建设技术债、事故、SQL 语料和 ADR 的索引，只沉淀试点中确实产生的经验。人员交接时要求接手者按照 handoff.md 恢复工作，并记录缺失资料；通过这些反馈完善模板，比由管理者凭想象添加大量字段更准确。

六十天验收不只看完成任务数量，还要看同一知识是否能够在另一条任务中减少错误或缩短定位时间。可以选择一次数据库方言问题，让第二名工程师仅依据专家库和案例找到适用边界，再用测试验证结论；如果条目无法说明版本或证据，就回到维护环节修订。团队开始试用事实日报和自动整理，但仍由负责人核对进展。统计工具等待、验证失败、权限问题与返工的主要来源，选一两个高频问题改进。闭环的定义是任务产生证据、证据更新知识、知识指导后续任务，三者有明确链接并能够被抽查。

12.4 第六十一至九十天：验证可复制，再决定推广

第三阶段选择第二个小团队或相邻模块进行复制。给他们经过试点验证的目录、模板、示例任务和启动 Prompt，同时保留修改模板的空间，让新团队根据真实语言、构建工具和发布方式调整。迁移时使用同样的验收指标，对比恢复能力、总交付时间、质量与成本，不要求所有项目强行采用 SQL 平台的目录细节。组织层面明确允许的工具、资料使用范围、维护职责、升级与异常处理流程，并把职责落在已有岗位上。推广目标是让多数任务能够按规范运行，同时把需要专家判断的情况及时暴露。

九十天结束时形成推广决策，列出已证明的收益、尚未证明的假设、质量限制与下一阶段投入。若一个项目收益明显而另一个项目复核成本过高，应分析任务结构和基础工程差异，不把结果简单归结为员工是否愿意使用 AI。对于依赖外部数据库环境或大量历史知识的项目，先补充可验证底座，再扩大自动化。把版本核验、知识维护、评估与培训纳入周期性工作，指定明确频率和维护人。企业最终需要的不是一次性部署完成的口号，而是遇到新需求、工具变化和人员流动时仍能继续运作的研发习惯。

12.5 第一天完整演练：六个步骤逐一走完

第一天的练习应当把一个最小需求走到可交接状态，而不是追求建立最多的文件或运行最多的模型。下面六个步骤依次准备项目、核实事实、形成任务包、完成最小实现、执行验证并交接结果。练习开始前，选择隔离的练习分支，安排一位开发负责人和一位验证者，约定本次只处理一个规则或一个结果字段。每完成一步，都检查相应文件是否写入实际证据，并确认下一位参与者可以据此继续。遇到工具不可用或信息缺失时，将原因写进状态记录，缩小练习范围；不要把未执行的测试标为通过，也不要为了凑齐流程而编造数据库支持结论。

步骤一：准备仓库与教学数据

创建或选择 sql-audit-platform 教学仓库，确认包含 pom.xml 和标准 Java 源码目录，把 README.md 中的 JDK、Maven 与启动要求逐项对照本机环境。第一次练习使用合成表结构与 SQL，明确项目是教学假定，不连接生产数据库，不带入客户日志。运行现有验证并记录实际结果；没有现成测试时先写明缺口，由熟悉项目的人建立一条最小测试后再开始功能练习。检查当前分支与未提交修改，保留可恢复的起点。此步骤的产物是明确的工作目录、环境记录和基线，而不是一段“环境应该没问题”的聊天承诺。

步骤二：建立规则入口与目录

在根目录创建 AGENTS.md，写入项目语言、代码路径、验证入口和任务文件读取要求，再创建 .ai/tasks/GAUSS-001、.ai/architecture/adr、.ai/knowledge、.ai/database、.ai/sql-case 与 .ai/prompts。依照前文模板填写背景、规则和流程，未知信息明确写为待确认。不要先填大量与当前项目无关的制度，也不要创建目录后就假定工具自动读取。开启首轮只读会话，要求 AI 报告实际根目录和说明来源，抽查它复述的规则。如果规则路径或内容不对，先修正打开方式与入口，再进入功能分析，避免错误上下文扩大到实现阶段。

步骤三：把需求写成可验收行为

填写 requirement.md，围绕“在教学 SQL 审核平台增加 GaussDB 类型入口与限定语法审核”说明用户输入、审核输出和暂不支持的行为。先确定产品形态与版本是否已知；不知道时记录待确认，并把真实数据库兼容验证留在未完成范围。至少准备正常输入、解析失败和不支持语法三个案例，说明每个案例应返回什么状态。让产品 AI 检查歧义并提出补充问题，负责人把已确定答案回写文件。完成标准是开发者可以依照案例判断功能是否正确，而不是需求里只出现“支持 GaussDB”一句宽泛目标。

步骤四：勘察与设计后再拆任务

使用第九章的勘察 Prompt 追踪真实审核链路，让 AI 列出实际入口、类型分派和规则实现，抽查文件与符号是否存在。根据现有结构填写 design.md，说明新增适配器如何接入、解析失败怎样表示、旧数据库怎样保持行为以及如何验证边界。存在重要取舍时创建提议 ADR，负责人完成审查后记录结论。再填写 plan.md，把任务拆成路由测试、适配实现、样本回归、评审与文档几个可验证切片。每个切片都有明确文件范围和完成证据，实施会话就能依顺序推进，也能在中断后知道从哪里恢复。

步骤五：实现、验证与评审

让开发 AI 从最小切片开始修改，完成后先查看差异，检查是否夹带无关重构、未解释的依赖升级或默认通过逻辑。运行实际约定的验证命令，把结果写入 test-report.md；失败时保留日志和原因，修复后针对相关范围复验。把需求、差异和报告交给评审角色，要求发现附上触发 SQL、预期与实际结果，不接受只有笼统建议的结论。重要问题修复后更新 review.md 与验证记录。此步骤可以只完成一个小切片，但必须完整展示实现与证据的连接，练习者才能学会把 AI 输出变成可评审的软件变更。

步骤六：交接并开启一次恢复演练

更新 status.md 和 handoff.md，记录当前提交、工作区差异、已完成行为、最后验证、未完成事项与第一条下一步操作。提交任务相关代码与记录后，换一个新会话或请另一名工程师读取任务目录，要求其复述当前状态、指出需要重新核验的条件并提出下一步。把恢复中发现的遗漏回写模板，例如测试报告没有提交标识、设计没有说明异常状态或计划缺少样本路径。第一天不需要上线生产，也不要求完成数据库全部适配；演练的验收是另一人能够依据记录继续工作，并知道哪些事实已有证据、哪些仍需验证。

12.6 排错、培训练习、FAQ 与总启动 Prompt

培训应让学员操作真实材料并发现错误。安排四种练习：从任务文件恢复一个中断会话，找出故意写错的根目录或规则来源，给缺少失败样本的审核规则补充案例，以及审查一份把环境失败写成测试通过的报告。每次练习要求提交可核验产物，例如修订的状态文件、测试路径和评审发现；讲师按事实、范围、证据和恢复能力评价。不要用“谁的 Prompt 最长”或者“谁生成的文件最多”作为成绩。熟练者的特征是知道先检查什么、何时需要专家证据，以及怎样把结论保存为下一人能够使用的资料。

表 12-2 常见排错场景与处理次序

现象	优先检查	处理与完成证据
AI 引用了另一个项目	根目录、分支、同名副本	切到正确目录，重新报告路径
AGENTS 规则不一致	覆盖文件、启动目录、文件内容	核对有效规则并记录来源
Maven 依赖下载失败	网络、镜像、代理、版本	修复环境后重新执行，不能报通过
只有新样本通过	旧数据库与异常样本是否运行	补充相应回归并保存结果
解析失败返回安全	异常分支与结果契约	加入复现样本、修复并复验
换电脑无法继续	提交是否推送、本地配置、交接记录	核对基线、恢复环境、说明下一步
资料互相矛盾	版本、日期、证据、替代状态	标记冲突并安排核验，更新主条目

常见问题一：是否必须建立六个独立 Agent？不必须，先由一个负责人按角色分阶段工作，只有并行或独立审查确实有价值时才增加会话。问题二：有了 AGENTS.md 是否就不用提示词？仍需要给出本次任务、范围与验收目标，规则文件只是稳定约定。问题三：AI 说测试通过是否可以合并？应核对真实命令、提交、报告和未覆盖范围，再按团队门槛处理。问题四：没有真实 GaussDB 环

境能否练习？可以练习接口、路由、静态样本与任务协作，但真实兼容性必须保持待验证。问题五：知识库越来越大怎么办？索引、状态、适用范围和归档比继续把全部内容送入会话更有效。

下面总启动 Prompt 可以保存为 .ai/prompts/bootstrap.md，首次使用时替换任务编号与真实项目情况。它把勘察、范围、实施、验证和交接串成一个协议，但不把尚未获得的权限或尚未确认的事实写成自动允许。团队可以删改措辞，保留五个基本要求：先核对工作对象，读取必要资料，区分证据与假设，完成与范围相符的验证，留下可恢复记录。当任务存在产品版本或真实环境缺口时，先推进不依赖缺口的部分，同时明确哪些验收仍待外部条件。这个模板的价值是约束工作行为，而不是保证任何工具都以同样方式自动执行所有文件。

```
总启动 Prompt (保存到 .ai/prompts/bootstrap.md) :
你协助维护本仓库。先核对项目根目录、分支、基础提交和未提交修改，说明工作区中已有变更的归属。
列出实际采用的说明来源；按 AGENTS.md 读取 .ai/AI_CONTEXT.md、.ai/AI_RULES.md、.ai/AI_WORKFLOW.md。
当前任务：GAUSS-001。读取 .ai/tasks/GAUSS-001 的 requirement.md、design.md、plan.md、status.md 和
handoff.md。缺失文件明确报告。
先勘察实际代码与测试，给出调用链、文件路径、事实、假设、未知项和最小改动范围；教学示例不得写为真实项目事实。
按计划推进可验证切片。范围变化、重要架构取舍、依赖或权限变化需要明确说明，并遵守已有授权与团队制度。
修改后检查差异，执行与变化相符的真实验证；未执行、被跳过和环境失败单独记录，不得称为通过。
更新 test-report.md、review.md、status.md、handoff.md；有可复用经验时更新知识索引或 ADR。
交付行为变化、实际验证、未完成项、风险与下一步。发布按团队明确的授权和流程执行。
```

附录 官方资料与模板索引

A 官方资料与核对范围

以下资料用于核对工具入口、数据库适用范围和 CI 配置。正文中的方括号标记对应本索引；企业使用时应按当前安装版本和运行环境复查。访问日期为 2026 年 10 月 11 日。流程模板、角色分工和试点指标属于本手册给出的实践建议，不能替代真实仓库、数据库日志与测试证据。

[S1] OpenAI 官方 AGENTS.md 指南
https://learn.chatgpt.com/docs/agent-configuration/agents-md

[S2] OpenAI 官方 Codex IDE 指南
https://learn.chatgpt.com/docs/codex/ide

[S3] OpenAI 官方 Git worktrees 指南
https://learn.chatgpt.com/docs/environments/git-worktrees

[S_GAUSS_INSTANCE] 华为云：GaussDB 数据库实例类型（集中式与分布式）
https://support.huaweicloud.com/intl/zh-cn/productdesc-gaussdb/gaussdb_01_013.html

[S_GAUSS_COMPAT] 华为云：GaussDB 兼容性说明
https://support.huaweicloud.com/productdesc-gaussdb/gaussdb_01_699.html

[S_GAUSS_PARAMS] 华为云：平台和客户端兼容性（集中式 V2.0-10.x）
https://support.huaweicloud.com/centralized-ref-v10-gaussdb/gaussdb-38-1043.html

[S_GH1] GitHub 官方：Workflow syntax for GitHub Actions
https://docs.github.com/en/actions/reference/workflows-and-actions/workflow-syntax

[S_GH2] GitHub 官方：Use GITHUB_TOKEN for authentication in workflows
https://docs.github.com/en/actions/tutorials/authenticate-with-github_token

[S_GH3] GitHub 官方动作仓库：actions/checkout
https://github.com/actions/checkout

[S_GH4] GitHub 官方动作仓库：actions/setup-java
https://github.com/actions/setup-java

B 核心模板与保存位置

表 19 任务与知识文件的快速索引

文件或目录	保存位置	主要用途
AGENTS.md	项目根目录	AI 入口与读取规则
AI_CONTEXT.md	.ai/	项目事实与边界
AI_RULES.md	.ai/	工程约束与变更规则
AI_WORKFLOW.md	.ai/	阶段产物与门槛
requirement.md	.ai/tasks/GAUSS-001/	范围与验收
design.md 与 plan.md	.ai/tasks/GAUSS-001/	方案与任务依赖
status.md	.ai/tasks/GAUSS-001/	状态与恢复
test-report.md 与 review.md	.ai/tasks/GAUSS-001/	测试审查证据

文件或目录	保存位置	主要用途
handoff.md	.ai/tasks/GAUSS-001/	交接与后续
ADR	.ai/architecture/adr/	决策与替代方案
数据库档案与 SQL 案例	.ai/database/ 与 .ai/sql-case/	版本边界与反例

使用索引找到对应章节后，先读模板前后的说明，再复制完整片段。创建新任务时替换 GAUSS-001，但保留同一组文件名和引用关系；完成后将需求、设计、测试和交接关联到同一个代码提交，知识库只保存已经核实的长期信息。目录条目、页码和正文导航由 Word 标题与目录域维护。